Adaptive Operator Selection for Meta-Heuristics: A Survey

Jiyuan Pei, Yi Mei, *Senior Member, IEEE*, Jialin Liu, *Senior Member, IEEE*, Mengjie Zhang, *Fellow, IEEE*, and Xin Yao, *Fellow, IEEE*

***Abstract*—Appropriate selection of search operators plays a critical role in meta-heuristic algorithm design. Adaptive selection of suitable operators to the characteristics of different optimization stages is an important task that owns promising potential to improve the performance of a meta-heuristic algorithm. A variety of adaptive operator selection methods have been proposed in last decades, from the machine learning and optimization communities. However, the existing studies have not been systematically reviewed so far. To fill the gap, this article provides a comprehensive survey of adaptive operator selection for meta-heuristics. According to the information required for selection, adaptive operator selection methods are classified into two categories: 1) stateless methods; and 2) state-based methods. Each category is further summarized into several key components. The strategies of each component belonging to the two categories are reviewed respectively. The motivation, strengths and weaknesses of the proposed strategies are also discussed. Furthermore, studied meta-heuristics and optimization problems in the literature are summarized. The effects from the difference of meta-heuristics and problems to the specific design of methods are discussed, together with the guidance of selecting the suitable method in different application scenarios. At the end, emerging challenges that could guide further research are discussed.**

***Impact Statement*—This article presents a lucid overview of existing adaptive operator selection methods, summarizes the design conception of each method and outlines avenues for future research development. Additionally, this article summarizes the meta-heuristics and optimization problems to which each adaptive operator selection method has been applied, highlighting their key characteristics that influence operator selection, and offers guidance on selecting the appropriate method for specific application needs. These insights benefit researchers seeking to leverage adaptive operator selection technologies to enhance optimization performance.**

***Index Terms*—Adaptive algorithm management, adaptive operator selection, automatic algorithm configuration, dynamic operator selection, selection hyperheuristics.**

I. Introduction

META-HEURISTIC algorithms have been widely used for solving complex optimization problems typically with huge solution space and many local optima [1]. Specifically in this article, *meta-heuristic* refers to optimization algorithms that conduct search by iteratively improving solutions with a set of rules or mathematical equations [1], namely *search operators*. First, one or multiple incumbent solutions are initialized randomly or by some heuristics [2]. Interactively, one or multiple new solutions are generated from the incumbent solutions by some search operators, and replace the incumbent solutions based on some acceptance criteria. With a proper balance between exploration and exploitation, meta-heuristics can search in the huge and multimodal solution space effectively, jump out of poor local optima, and reach near-optimal solutions in a reasonably short time. Common examples of meta-heuristics include genetic algorithm (GA) [3], particle swarm optimization (PSO) [4], tabu search [5], variable neighborhood search (VNS) [6], evolution strategy (ES) [7] and simulated annealing (SA) [8].

For a specific family of problems, an efficient algorithm requires a good algorithm configuration, which includes search operators, selection mechanisms and appropriate parameters [9], [10], [11], [12]. Focusing on search operators,[1] different operators are preferred for searching different regions in the solution space [13]. Consequently, the choice of the most suitable operator shifts as the search progresses through the solution space. Focusing on search operators, two main tasks arise: 1) designing a wide range of operators that can play different roles during the search; and 2) taking advantage of these operators smartly so that the most appropriate operator can be used

[1]From now on, unless stated otherwise, *operator*s refers to search operators.

Received 30 June 2024; revised 25 October 2024 and 1 January 2025; accepted 16 February 2025. Date of publication 25 February 2025; date of current version 31 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62250710682, and in part by Guangdong Major Project of Basic and Applied Basic Research under Grant 2023B0303000010. This article was recommended for publication by Associate Editor Zhi-Hui Zhan upon evaluation of the reviewers' comments. *(Corresponding author: Yi Mei.)*

Jiyuan Pei is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China, and also with the Centre for Data Science and Artificial Intelligence & the School of Engineering and Computer Science, Victoria University of Wellington, 6012 Wellington, New Zealand (e-mail: jiyuan.pei@vuw.ac.nz).

Yi Mei and Mengjie Zhang are with the Centre for Data Science and Artificial Intelligence & the School of Engineering and Computer Science, Victoria University of Wellington, 6012 Wellington, New Zealand (e-mail: yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

Jialin Liu is with the School of Data Science, Lingnan University, Hong Kong SAR, China (e-mail: jialin.liu@ln.edu.hk).

Xin Yao is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China, and also with the School of Data Science, Lingnan University, Hong Kong SAR, China (e-mail: xinyao@ln.edu.hk).

This article has supplementary downloadable material available at https://doi.org/10.1109/TAI.2025.3545792, provided by the authors.

Digital Object Identifier 10.1109/TAI.2025.3545792

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

for searching the corresponding regions in the solution space. With regard to the former task, a variety of problem-specific operators has been designed (e.g., [10], [14], [15]). In contrast, the latter is much harder to achieve. Adaptively selecting the best operator for a meta-heuristic during its search process is called *adaptive operator selection* (AOS).

Here we formally define the task of AOS as follows: given an optimization problem instance (a solution space $\mathcal{X}$ and an objective function f) and a meta-heuristic, including the initialized solution(s) $X_0 \subset \mathcal{X}$, a pool of operators $\mathcal{O}$, a maximal number of iterations $T \in \mathbb{Z}^+$, a solution update procedure $X' \sim \mathcal{D}_\Theta(X, o, t)$, where $\mathcal{D}_\Theta(X, o, t)$ represents the probability distribution (parameterized by Θ) of new solution(s) X' conditioned on the incumbent solution(s) $X \subset \mathcal{X}$ and operator $o \in \mathcal{O}$ at iteration t, and a solution output method $x^* = U(\Omega)$ that outputs the final solution from all the examined solutions $\Omega \subset \mathcal{X}$ by the meta-heuristic, the task of AOS is to select the optimal operator sequence $\mathbf{o}^* = (o_1^*, o_2^*, \ldots, o_T^*) \in \mathcal{O}^T$ to minimize[2] the expectation of the objective value of the final solution $f(x^*)$, i.e., $\min_{o_1,\ldots,o_T \in \mathcal{O}} \mathbb{E}[f(x^*)|x^* = U(\{X_0, X_1, \ldots, X_T\})]$, where $X_{t+1} \sim \mathcal{D}_\Theta(X, o_t, t)$. Note that it can be extended to a set of problem instances, where the objective is the expectation of the (normalized) objective values over all the instances.

For solving a given problem, different operators have different performances in different situations, and adaptively selecting suitable operators during optimization is crucial for the meta-heuristic. For example, large-step operators are needed to effectively escape from the current region of the solution space when the search is stuck in a local optimum (i.e., exploration), while small-step operators are needed to encourage the exploitation of a newly reached local area to find solutions of better quality. Compared with commonly adopted methods, e.g., random selecting or using operators in turn, effective AOS methods can boost the efficiency of meta-heuristics in solving optimization problems, especially when the candidate operator set owns high diversity. For a given set of problems, it is important and desirable to design the optimal AOS method to maximize the effectiveness of operators.

The concept of AOS origins from the literature of adaptively adjusting the probabilities of using operators (e.g., [16], [17], [18], [19], [20]). Since then, investigations into the adaptive selection of operators have been recognized (e.g., [10], [21]). AOS is sometimes identified as belonging to *adaptive parameter tuning* [22] or *selection hyperheuristic*s (SHHs) [23], where each candidate operator is considered as a possible parameter setting or a heuristic strategy, respectively. There are studies specifically focused on adaptive selection of neighborhood search operators in large neighborhood search (LNS) algorithm [24], [25], [26], termed as *adaptive large neighborhood search* [27]. AOS is also related to *automated algorithm selection* [28] and *algorithm portfolio* [29], [30], [31], [32]. While AOS concentrates on selecting effective operators within a single meta-heuristic during problem-solving, automated algorithm selection and algorithm portfolio approaches primarily emphasize selecting suitable optimization algorithms, each capable of independently solving the problem. Reinforcement learning (RL) [33] has been applied to automated algorithm selection [28] and neural combinatorial optimization [34]. It is also often used as a tool to learn to select operators [35], [36], [37], [38].

To our best knowledge, there is no existing survey in AOS for meta-heuristics. Some studies [39], [40], [41], [42] conduct limited comparison or classification of specific AOS methods, focusing on particular AOS components [39], specific meta-heuristics [40], [42], or solving specific problems [41]. In contrast, this survey provides a comprehensive review of all AOS studies, without limiting the components, meta-heuristics, or problems. Some surveys discussed about topics related to AOS, including the recent survey of SHHs [23], the survey of machine learning in combinatorial optimization [35], and the survey of adaptive large neighborhood search [27]. However, they cover only limited AOS studies that overlap with their topics, leaving out other AOS research. In contrast, our survey encompasses all studies within the scope of AOS.

AOS is crucial for both academic and practical reasons. Academically, it addresses the fundamental question of how to efficiently allocate computational resources during optimization. Practically, meta-heuristics have been successful in numerous real-world applications [14], [43], making improvements of meta-heuristics by AOS highly valuable. This motivates the need for a comprehensive survey of AOS for meta-heuristics, benefiting both researchers and practitioners.

The contributions of this survey are

1) We provide a formal and general definition of the AOS task.
2) We propose a comprehensive taxonomy of AOS methods that encompasses all existing related studies.
3) We summarize and discuss the advantages and limitations of existing AOS methods, offering guidance for selecting AOS methods for practical applications and informing future research in AOS method development.
4) We highlight the applications of AOS across various meta-heuristics and optimization problems, discussing specific concerns relevant to particular problem-solving contexts.

The rest of this article is organized as follows. Section II describes the scope of this survey. Our taxonomy of AOS is proposed in Section III. Sections IV and V review the two categories of AOS methods according to our taxonomy, i.e., stateless and state-based methods. Section VI reviews the optimization problems and meta-heuristics studied in the literature, together with the AOS method selection guidance for practical needs. Limitations of existing methods and challenges that could guide future research are discussed in Section VII. Finally, Section VIII concludes the article.

II. Scope of Review

In this survey, we collected all the relevant articles by searching in Google Scholar, Web of Science and IEEE Xplore with various search terms including "adaptive operator selection,"

[2]Without loss of generality, this article assumes minimization problems.

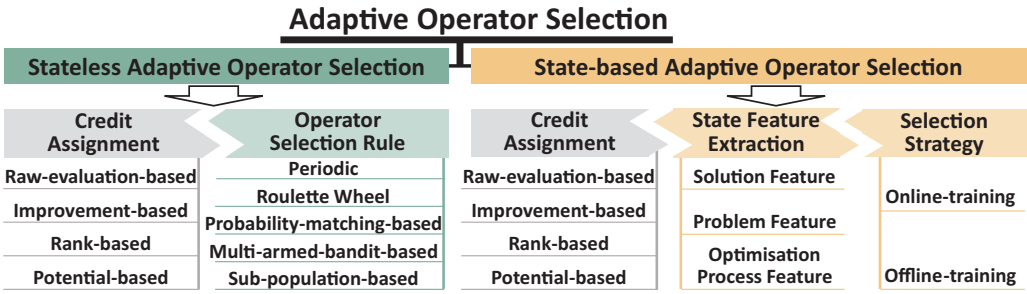


Fig. 1. Taxonomy of AOS methods. The unique components of stateless AOS methods are highlighted in green, while those of state-based AOS methods are highlighted in yellow. The common component (credit assignment) is highlighted in gray. In both stateless and state-based AOS methods, the component divisions are orthogonal, enabling any combination of instances of these components to create an instance of a corresponding AOS method.

"dynamic operator selection," "adaptive operator probability," "adaptive operator scheduling," ("hyperheuristic" and "operator selection"), ("automated algorithm design" and "meta-heuristic"), "adaptive heuristic selection", and ("case-based reasoning" and "optimization"), by February 2024. The following criteria are used to determine the (ir)relevance of an article to our survey.

1) Any work that adaptively selects operators during the search process of meta-heuristics is considered relevant and included in the survey.
2) Methods that select operators primarily based on *hand-crafted* rules and human knowledge of specific operators' and meta-heuristics' characteristics, while demonstrating strong performance in certain scenarios (e.g., [44], [45]), are not considered as AOS methods and are therefore excluded from this survey.
3) For adaptive operator parameter selection, if the parameter domain is predefined and enumerated with a finite number of possible values to be automatically selected, then the methods are considered relevant to AOS, and included in the survey.

III. Taxonomy

In general, an AOS method selects an operator following a probability distribution over all candidate operators, called a *selection probability distribution*. The operators' performance, and consequently the optimal selection probability distribution, shifts as the search progresses through the solution space. When adaptively selecting operators, some AOS methods consider the current optimization state (e.g., locations of the incumbent solutions in solution space), while others do not. Handoko et al. [46] for the first time uses the term *stateless* to describe the methods in which the operator selection does not rely on the current optimization state. Following this perspective, we categorize existing AOS methods into two classes: *stateless* and *state-based* methods. The key difference between the two classes is whether the current optimization state is a mandatory input for selecting an operator or not.

Stateless methods capture the changes by assessing operators' performance after applying them. Therefore, those methods make decisions solely based on the empirical performance of operators. Typically, a stateless AOS method is composed of two primary components [13]: *credit assignment* (CA) and *operator selection rule* (OSR). CA evaluates the contribution of applying an operator as a reward value (i.e., credits), while OSR selects the operators based on the rewards. In other words, OSR decides how the selection probability distribution is computed based on the credits determined by CA, and how operators are selected given the distribution.

In addition to credits, state-based methods take into account the current *optimization state*. In the literature, optimization states are often described with the characteristics of the solution region being searched (e.g., the statistics of decision variables and their objective evaluations), explicitly formulated as *state features*. Learning models trained with state features and credits can be used to predict operators' performance prior to their application and assist the operator selection. State-based AOS methods are composed of: 1) *CA*; 2) *state feature extraction*; and 3) *selection strategy*. State feature extraction determines the features representing an optimization state, while selection strategy contains a learning model, a model training method, and a decision strategy which selects an operator according to model predictions. In this article, when discussing state-based AOS, we primarily focus on model training, as this is where the main challenge introduced by AOS lies. Model design and decision strategy largely follow methods from the field of machine learning.

The CA-OSR taxonomy [13] is mainly for stateless methods, and cannot be employed in state-based methods. To better illustrate the process of general AOS, this article proposed a new taxonomy of AOS methods, extending from the CA-OSR taxonomy. The proposed taxonomy categorizes AOS methods according to their components. Consequently, various AOS methods have been designed in the literature. The strategies for each component are further classified into multiple categories, forming a general and modularized taxonomy (cf. Fig. 1).

IV. Stateless Adaptive Operator Selection

Stateless AOS methods are the most commonly studied in the literature. Those methods merely assume that the effectiveness of operators is relatively stationary and changes smoothly along the search process. If an operator performs well at the current iteration of optimization, its performance is unlikely to drop suddenly at the next iteration.

Algorithm 1: Meta-Heuristic Framework with Stateless AOS.

The unique steps of stateless AOS are highlighted in green, and the common step of stateless and state-based AOS are highlighted in gray.

$x \leftarrow$ initialize solution
$y \leftarrow f(x)$ ▷ Evaluate solution
$\pi \leftarrow$ initialize selection probability distribution
for $t \leftarrow 1$ to T **do**
  $ope^* \leftarrow \pi()$ ▷ Select operator
  $x' \leftarrow ope^*(x)$ ▷ Apply selected operator
  $y' \leftarrow f(x')$ ▷ Evaluate solution
  if $y' < y$ **then**
    $x \leftarrow x'$, $y \leftarrow y'$
  end if
  $r \leftarrow CA(x, x', y, y')$ ▷ Calculate reward by credit assignment
  $\pi \leftarrow OSR(\pi, r, ope^*)$ ▷ Update π by operator selection rule
end for
return x

Algorithm 1 demonstrates a typical meta-heuristic framework with stateless AOS, in which AOS steps are highlighted. At each iteration, one or multiple operators are selected to generate one or multiple new solutions from the incumbent ones. Subsequently, the newly generated solutions are evaluated. A set of solutions is selected to replace the incumbent solutions according to some criteria. CA calculates the rewards of using those operator(s), which are then used by OSR to update the probability distribution of selecting each operator for the next iteration. The above process repeats until certain stopping criteria are met (e.g., reaching the maximal iteration number T). Finally, the best-so-far solution is returned.

Note that meta-heuristics with operator selection in this framework can be either a single-solution-based search (e.g., local search [47]) or a population-based search (e.g., GAs [48]). The following sections review the strategies designed for the CA and OSR modules, respectively.

A. Credit Assignment (CA)

CA plays a crucial role in stateless AOS, as it evaluates the contribution of operators and is responsible for identifying *good operators* that are desired. Ideally, an operator should be evaluated based on its contribution to the quality of the final solution(s) obtained at the end of optimization process. The *reward of an operator given an incumbent solution* can be defined as the expected final solution quality obtained by the search subsequent to applying the operator to the incumbent solution. Then, if multiple incumbent solutions exist, obtaining the reward of an operator requires finding all expected final solutions based on all incumbent solutions. At each iteration, an AOS method evaluates candidate operators with its CA method and selects the operator with the highest potential of obtaining the largest improvement in the final solution quality.

In practice, it is hard to accurately measure the contribution at each iteration within reasonable computational cost, which requires running the optimization towards the end to obtain the final solution(s). Thus, the actual reward is unattainable. Existing CA methods mainly estimate an operator's contribution from its immediate utility based on the generated solutions. Furthermore, the contribution of an operator depends on the incumbent solution it manipulates. When characteristics of incumbent solutions change massively after an iteration, the potential contribution of operators changes correspondingly. It is sometimes necessary to reevaluate all the candidate operators after each iteration, to better track the changes. It is challenging to design an accurate and efficient CA method.

TABLE I
SUMMARY OF CA METHODS AND RELATED PAPERS CATEGORIZED ACCORDING TO THEIR MAJOR NOVELTY

Category	Publications
Raw-evaluation (Section IV-A1)	Raw fitness [49], [50], [51], [52], [53]
Improvement (Section IV-A2)	FI [21], [54], [55], [56]; Re-scaled FI [48], [57], [58], [59], [60], [61], [62]; FI with sliding window [41], [63], [64], [65], [66], [67], [68], [69], [70], [71], [72], [73], [74]; Hyper-volume improvement [75]; FIR [76], [77], [78], [79], [80], [81]; Extreme/average FIR [47], [82]; FRR [40], [76], [77], [83], [84], [85], [86], [87], [88], [89]; Success [90], [91], [92], [93], [94], [95], [96], [97]; Success rate [77], [96], [98], [99]; Cumulative Success rate [100], [101], [102], [103], [104]
Rank (Section IV-A3)	Sum of ranks [105]; Fitness-based area under the curve [67], [105], [106], [107]; Survival [108], [109], [110]; Survival rate [111], [112], [113], [114], [115]
Potential (Section IV-A4)	Weighted sum of diversity and fitness [116], [117]; Dominance based on fitness and diversity [118], [119], [120], [121], [122]; Compass [123], [124]; Extreme/average compass [125], [72]; FLA metrics [98], [126], [127], [128], [129], [130]

Note: FI: fitness improvement, FIR: fitness improvement rate, FRR: fitness-rate-rank, FLA: fitness landscape analysis.

Various CA methods have been proposed in the literature, with different focuses for estimating the performance of operators. To better illustrate the focus and conception of design of each CA method, we categorize those methods into the following four classes. *Raw-evaluation-based CA* methods (cf. Section IV-A1) directly use the objective (or fitness) values of generated solutions as the reward. *Improvement-based CA* methods (cf. Section IV-A2) formulate reward based on the improvement of objective values. *Rank-based CA* methods (cf. Section IV-A3) rank all newly generated solutions, and then reward operators based on the ranks of their generated solutions. *Potential-based CA* methods (cf. Section IV-A4) reward operators according to their potential, measured by indicators like population diversity. Table I summarizes the CA methods described in this section and categorizes related papers according to their major novelty.

1) Raw-Evaluation-Based Credit Assignment: The general idea of raw-evaluation-based CA methods is assigning higher rewards to operators that generate solutions with higher objective (or fitness) values. The evaluation value of the new solution,

is the most straightforward definition of reward and often used in earlier studies (e.g., [49], [50], [51], [52], [53]). However, it rewards operators even if no improvement was made.

2) Improvement-Based Credit Assignment: Improvement-based CA is the most widely studied category of CA. Its general idea is assigning higher rewards to operators that achieve larger improvement in solution fitness. Based on this idea, different reward formulations have been proposed.

Fitness improvement (FI) is widely used as reward, encouraging using operators that obtain larger absolute fitness improvement [21]. Given an incumbent solution x and a new one x'_i obtained by applying the operator i to x, FI takes the difference between $f(x)$ and $f(x'_i)$ as the reward of the operator i, formulated as (1) following [13]:

$$FI_i = \max\{0, f(x) - f(x'_i)\}. \tag{1}$$

When multiple solutions (denoted as $x'^1_i, x'^2_i, \dots$) are generated by one operator i or multiple incumbent solutions (denoted as $x^1, x^2, \dots$) are manipulated (e.g., crossover operators in GA), the averaged fitness of generated solutions [56] or the best fitness of incumbent solutions [59], [61] are commonly used for FI calculation, respectively.

Furthermore, FI values can sometimes vary significantly in successive iterations, with the improvement of solution quality. The average of FI values recorded from historical iterations can be used as reward for more stable values. In [41], [66], [69], a fixed-size sliding window records the most recently obtained FI values of an operator, and the average value of the window is then used to evaluate operators. The study by [63] proposed to take the *extreme* (maximal) value in a sliding window reward to assign more attention to operators that achieve sparse but greater improvements, to capture the breakthrough in solution quality during optimization. Another approach is rescaling FIs, formulated in reference to the fitness of representative solutions such as the best fitness in the current population [48], [58], [60] or the best-so-far fitness [62]. For example, fitness improvement rate (FIR) [76] is a common paradigm of normalized FI [77], [78], [79], [80], [81], [131], taking the fitness of the manipulated incumbent solutions selected to generate new solutions as the reference, demonstrated as

$$\text{FIR}_i = \frac{f(x) - f(x'_i)}{f(x).} \tag{2}$$

FIR can also adopt the average [82] and extreme values [47] within a sliding window.

Involving operators' ranks in the sliding window, fitness-rate-rank (FRR) [76] is proposed based on FIR. The historical ranks of operators are used to discount the rewards, so that a worse-performed operator with a lower rank obtains a significantly lower reward than promising operators.

There are also some specific designs of improvement-based CA for handling specific characteristics of optimization problems. For multiobjective optimization problems that require simultaneously optimizing multiple contradictory objectives, multiobjective indicators, such as hypervolume, are used to replace fitness for quantify improvement [75], [132]. By decomposing a multiobjective problem into multiple single-objective subproblems, FI and its variants can be directly applied to each subproblem [66], [69]. The work of [133] formulated the reward based on the increase in the rate of feasible solutions in the population and the degree of constraint violation when handling constrained optimization problems.

Some works also took into account the computational cost of operators. For example, in [20], [134], the FI gained per time unit or solution evaluation is taken as the reward value. The work of [135] proposed the choice function method, where the an operator is evaluated by the weighted sum of FI of the operator gained per time unit, FI for consecutive pairs of operator gained per time unit, and time elapsed since the operator was last applied. The choice function method and its variants are widely studied in SHH research, with more details and discussions available in the recent SHH survey [23].

Another type of method, called *success*, rewards an operator with a predefined value (e.g., 1) as long as it achieves success in improvement, regardless of the actual improvement degree. Success methods differ in the reference solution and the definition of success. A common definition of success is if an operator generates a solution with better fitness than the manipulated incumbent solution [90], [91], [92], [93], [94], [95], [97]. The best solution in the current population is also used as the reference solution [136]. Considering constrained optimization problems, the improvement of solution constraint satisfaction [93] and the discovery of an unseen solution [91] were also defined as success. The work of [90] defined success as the improvement of either the overall average error or the error on any single test data point in genetic programming algorithms for solving feature construction problems. In [77], the number of successfully improved objectives is used as the reward, for solving multiobjective continuous optimization problems. The Pareto dominance relations are used to define success when multiple objective functions exist to evaluate a solution [96]. The generation of a new solution that dominates the incumbent one is considered a success. When multiple solutions are generated by a single operator in an iteration, the ratio of success in reference to the total number of generated solutions (namely *success rate*) can be used as the reward value [98], [99], [137]. Maintaining a sliding time window to record the success in recent iterations, cumulative success rate [100], [101], [102], [103], [104] have also been defined.

Improvement-based CA methods are often intuitive and have several advantages. The formulations of reward can solely rely on solution evaluation, with the option to include additional instructive information if needed. Consequently, such methods are easy to implement and the computational complexity is relatively low. However, directly taking the improvement value as the reward suffers from two issues: 1) diminished improvement, i.e., the improvement tends to slow down during optimization; and 2) oscillating reward, i.e., the improvement achieved by the identical operator on different incumbent solutions varies greatly, even within the same iteration. Normalization methods can mitigate the former issue, but finding a universally effective normalization reference is challenging. The oscillating reward may lead to noisy performance estimation and severely impair the performance of AOS [138]. When the variance of rewards

for each operator exceeds the expected difference between them, it becomes difficult to select the true best operator. Sliding windows are commonly used to stabilize reward values, but this can reduce sensitivity to immediate changes in operators' performance. Success methods reward operators with a predefined value when an improvement is achieved, disregarding the degree of improvement. Although the two issues are relatively less severe in success methods, success methods are less effective in highlighting the significance of performance differences between operators. Furthermore, improvement-based methods are shortsighted, as they only consider the immediate performance (e.g., the improvement in solutions' quality during a single iteration), without accounting for operators' long-term impact (e.g., potential to find better solutions in future iterations). Consequently, operators that contribute more to long-term optimization performance may be underestimated.

3) Rank-Based Credit Assignment: Instead of evaluating improvements based on incumbent solutions, rewards can also be defined by ranking newly generated solutions. These rankings are typically represented by positive integers $1, 2, 3, \ldots$. Operators that generate higher-ranked solutions obtain larger rewards. By using the solutions generated within the same iteration as references for each other, the rank-based method can better mitigate the instability of reward values, thereby more effectively evaluating the operators.

One common way is to rank new solutions based on their fitness. The work of [105] proposed two rank-based CA methods, i.e., *sum of ranks* and *fitness-based area under the curve*. The sum of ranks method uses the sum of ranks of all the generated solutions of an operator as the reward value, where better-quality solutions have larger rank integers. Compared with the sum of ranks method, the fitness-based area under the curve method assigns more attention to the higher-ranked solutions in terms of reward, as higher-ranked solutions may contribute more to directing the search [105].

Survival selection, another important component in meta-heuristics besides search operators, also conducts solution rank. After generating and evaluating all new solutions, survival selection methods choose desired solutions to preserve for the next iteration and discard the rest. These methods inherently involve ranking strategies, and only higher-ranked solutions are preserved. Sophisticated survival selection strategies have been designed to address specific problem difficulties, such as stochastic ranking-based selection for handling constraints in constrained optimization problems [139]. CA methods based on survival selection, namely *survival*, reward an operator if the generated solution is selected into the next iteration. By leveraging survival selections' capabilities, survival CA methods can effectively evaluate the performance of operators in addressing these specific problem difficulties.

The works of [108], [109], [110] calculated operators' reward with the results of nondominated sorting-based selection in nondominated sorting genetic algorithm (NSGA) to evaluate operators' performance through multiple objective functions simultaneously. The reward value of each operator is initialized as 0 at the beginning of an iteration. Each new solution that survives into the next iteration increases the reward of the operator that generated it by 1. However, an operator with a higher selection probability tends to obtain a higher reward as more new solutions are generated through it, disregarding its current performance. *Survival rate* [111] method handles the issue, by taking the proportion of newly generated solutions that survive as the reward of an operator. Survival rate is typically used with stochastic ranking-based survival selection [139] for solving constrained optimization problems [112], [113], [114], [115]. The survival method is similar to the success method (cf. Section IV-A2), as both involve the quality comparison between solutions. However, the latter rewards operators only when an improvement is achieved, while the former may reward operators even without improvement, as long as the generated new solution is selected into the next iteration.

By taking new solutions as references for each other and calculating rewards based on ranks rather than raw fitness values, rank-based methods achieve more stable reward values compared to improvement-based methods. However, the reward of an operator depends on the performance of other operators, even though they operate independently of each other. This dependency can complicate the detection of operators' performance changes. Furthermore, new solutions are generated within the same iteration by different operators, necessitating that all operators be applied in a single iteration and potentially increasing computational costs. Moreover, rank-based methods are also shortsighted, similar to improvement-based methods.

4) Potential-Based Credit Assignment: Statistics collected during optimization, e.g., population diversity in population-based meta-heuristics, can sometimes indicate the potential optimization performance in future iterations. The general idea of potential-based CA methods is to assign higher rewards to operators that own higher potential in future optimization process, estimated with past optimization records.

For population-based meta-heuristics, a better population diversity commonly indicates a better coverage of the search space and a higher chance of finding high-quality solutions in future iterations. Population diversity can be defined as the distances between solutions in the decision space. CA methods based on diversity give higher reward values to operators that achieve better diversity with different distance metrics. For continuous problems, Euclidean distance is commonly used to measure the solution diversity [116]. For combinatorial problems with different solution representations, designing suitable distance metrics is crucial. Hamming distance is commonly used for binary assignment problems (e.g., [72], [124]), where a solution is represented as a binary vector. For permutation problems, with solutions as sequences of elements, the proportion of common consecutive element pairs is used as the distance between solutions (e.g., [112], [130]).

The work of [124] proposed to reward operators by the weighted sum of diversity improvement and fitness improvement of the population, named *compass*. By changing the weights of the two terms, the method can balance between short-term evaluation and long-term evaluation and guide the optimization process towards different directions. Similar to extreme and average FI in a sliding window, average compass [125] and extreme compass [72] are also investigated. Not only

the weighted sum, dominance relation based on fitness and diversity can also be used in calculating the reward [118], [119], [120] in population-based meta-heuristics. In single-solution meta-heuristics like LS where the solution population is not available, solutions found in the past few iterations are recorded for diversity measurement [121], [122].

Apart from the decision space, features of objective space also reflect the potential search ability. In fitness landscape analysis (FLA) studies, many metrics have been proposed to evaluate the objective-related features [140], for example, information landscape [141] reflects the ability of an operator to move to a region with better solution quality [98]. With higher value of such metrics an operator provides better potential in the future. Various FLA metrics are investigated in solving continuous optimization problems, where the reward value is calculated commonly as the weighted sum of FLA metrics and improvement-based CAs, i.e., success rate [98], [128], [129] or FIR [127], so that both immediate and long-term performance are reflected. FLA metrics require a decent number of samples, i.e., fitness evaluations, to conduct a precise evaluation which makes them relatively computationally expensive.

Potential-based methods consider operators' long-term performance, enhancing the global search ability. However, designing suitable indicators poses significant challenges. These methods incur higher computational costs compared to improvement-based and rank-based methods.

B. Operator Selection Rule (OSR)

Based on the rewards obtained by CA, OSR provides the operator selection probability distribution for the next selection. In meta-heuristics, the performance of an operator, and consequently the optimal selection probability distribution, changes during the optimization process [13]. OSR is responsible for handling the dilemma between exploitation (i.e., using the estimated best operator for better optimization) and exploration (i.e., testing other operators to improve the estimation accuracy and track the performance change). We classify the commonly studied OSR methods into five classes, periodic selection (Section IV-B1), roulette wheel selection (Section IV-B2), probability matching (PM)-based selection (Section IV-B3), multiarmed bandit (MAB)-based selection (Section IV-B4) and subpopulation-based selection (Section IV-B5). Below we review each of the five classes.

1) Periodic Selection: Periodic selection methods explicitly split the optimization process into multiple exploration periods and exploitation periods. In the exploration period, all operators are selected in turn or uniformly randomly to equally test their performances and collect rewards. Then in the following exploitation period, the methods apply only the operator with the largest reward deterministically. Periodic selection methods majorly differ in the length and order of periods.

As the simplest method, all periods can have the same length (e.g., number of optimization iterations) and the two types of periods alternate iteratively [129]. However, the best operator may change before an exploitation period ends or remain unchanged through multiple exploration periods, leading to a waste of computational resources by exploiting nonoptimal operators or unnecessarily exploration. Research into dynamically controlling methods involves adjusting either the length of periods [86] or determining the type of the next period [77], [78], [82], aiming to initiate an exploration period only when performance changes are anticipated to occur.

Because all operators are used equally regardless of their performance in the exploration period, and each period is exclusively dedicated to either exploration or exploitation, periodic selection methods lack flexibility and cannot maximize the utilization of computational resources.

2) Roulette Wheel Selection: With the similar strategy to the roulette wheel solution selection method, operators can be randomly selected subjecting to their obtained rewards. Since each operator has a probability of being selected during every operator selection, each selection can serve either exploitation or exploration purposes flexibly. As a result, the optimization process no longer requires explicit division into distinct exploration and exploitation periods.

Typically roulette wheel operator selection methods assign the ratio of an operator's cumulative reward (i.e., the sum of all rewards received since the beginning of the optimization) over the sum of cumulative rewards of all candidate operators as its selection probability [57], [75], [91], [99], [115], [142], [143]. Additional mechanisms can be implemented to achieve specific trade-offs between exploration and exploitation. The upper and lower bound of cumulative reward were introduced to encourage exploring the operators with low rewards in [91], [115]. The work of [142] encouraged exploitation by alternately selecting deterministically the operator with the highest reward and randomly with the roulette wheel method.

Because the performance of operators may change over time, rewards from too long ago may be misleading for the current selection. Discarding earlier reward records (e.g., [94], [96]) may better accommodate changes in operators' performance than considering all rewards received.

3) PM-Based Selection: In [21], the probability matching (PM) method [144] was used for AOS. Each operator possesses a Q value to estimate its overall performance. Unlike the roulette wheel operator selection method, where historical rewards with the same value contribute equally to the selection probability, in PM, recently received rewards contribute more.

A formal formulation of the PM method for AOS was given in [54], as demonstrated in (3) and (4)

$$Q_i \leftarrow \alpha \cdot r_i + (1-\alpha)Q_i \tag{3}$$

$$P_i \leftarrow P_{\min} + (1 - K \cdot P_{\min})\frac{Q_i}{\sum_{j=1}^{K} Q_j} \tag{4}$$

where K is the number of candidate operators, r_i is the reward of operator i calculated by a CA method, Q_i is the performance estimation of operator i and P_i is the selection probability of operator i. $P_{\min} \in (0, 1)$ and $\alpha \in (0, 1]$ are two predefined parameters. $P_{\min}$ is needed to retain a minimal degree of exploration and avoid the exclusion of any operator, i.e., selection probability reduced to 0 (or very close to 0) due to temporarily low rewards. α controls the emphasis difference between recent

and earlier rewards, where a larger α value indicates greater emphasis on recent rewards.

Then adaptive pursuit (AP) is proposed [54] based on PM, for quicker probability distribution change when the performances of operators are changed, formulated as (5)

$$P_i \leftarrow \begin{cases} \beta \cdot P_i + (1-\beta) \cdot P_{\max}, & \text{if } i = \arg\max_j Q_j \\ \beta \cdot P_i + (1-\beta) \cdot P_{\min}, & \text{otherwise} \end{cases} \tag{5}$$

where $P_{\max} = 1 - (K-1) \cdot P_{\min}$ and $\beta \in (0, 1]$ is a predefined parameter. AP exploit more than PM, by making the selection probability of the estimated best operator converge to the predefined upper bound $P_{\max}$ and of other operators equally converge to the predefined lower bound $P_{\min}$ [54].

PM and AP are two of the most commonly used OSRs, applied on a variety of meta-heuristics, such as GA [63], [145], LS [70], [137], iterated local search (ILS) [102], [146], differential evolution (DE) [60], [66], memetic algorithm (MA) [59], [61], NSGA [108], [109], [110], and artificial bee colony (ABC) [68].

With specific preferences in the exploration–exploitation balance, modifications on PM and AP are designed, for example, directly assigning a predefined high selection probability to the estimated best operator (i.e., with the highest Q value) instead of adjusting from historical values (e.g., (4)) for more exploration [71], [100], and using uniform probability when no improvement is achieved for consecutive iterations to increase the exploration [147]. To better utilize multiple good operators instead of only the best one, the work of [137] proposed the generalized AP method, which allows assigning any predefined target probability for each operator to converge, instead of fixedly assigning $P_{\max}$ to the best operator and $P_{\min}$ to all the others. Methods that select and apply a sequence of operators instead of just one operator are also investigated to better utilize the cooperation between operators [93], [148].

PM-based methods effectively handle the changes in operators' performance by gradually reducing the contribution of earlier obtained rewards on the current selection probabilities. Additionally, the probability lower bound ($P_{\min}$) ensures exploration effectively. Nevertheless, extra parameters are introduced, which necessitates careful tuning.

4) MAB-Based Selection: As discussed in [13], AOS can be viewed as a special case of *multiarmed bandit* (MAB) problem [149]. As operators' reward distributions vary, the most related MAB variants to AOS include nonstationary MAB [150] and contextual MAB [151]. Algorithms developed for solving MAB problems are used for AOS.

Based on the upper confidence bound (UCB) [152], a classic algorithm for solving MAB, the study by [55] proposed the dynamic MAB (DMAB) as an OSR method. DMAB deterministically selects the operator with the highest UCB value [55]. In addition, the Page-Hinkley test [153] is embedded into DMAB to detect the abrupt changes in the performance of the operators [55]. Once an abrupt change in operators' reward is detected, the records of all the operators are cleared, and then DMAB reevaluates the reward of each operator from scratch.

Based on DMAB, the work of [65] proposed sliding MAB, in which a sliding window is maintained to record the recent Q values and the total application number of each operator for detecting the changes in any degree instead of only abrupt changes defined by the parameters of Page-Hinkley test. In [67], sliding MAB was used to adaptively select not only operators but also meta-heuristic algorithms. The window size can also be adaptively adjusted [154]. Other mechanisms are also designed to enhance DMAB in specific scenarios. In [119], DMAB was enhanced with a pre-selection mechanism to conduct selection from 307 available operators. The study of [123] parallelized the DMAB process to facilitate parallel operator application in parallel meta-heuristics.

In the study of MAB, variants of UCB, e.g., UCB-Tuned and UCB-V [152], are proposed and verified with high performance. The work of [84] investigated OSR based on UCB-Tuned and UCB-V. Experiment results demonstrate that UCB-Tuned performs better than UCB-V and classic UCB for operator selection on multiobjective continuous problems. Thompson sampling [155], another classic algorithm for solving MAB problems, and its variants are also investigated as OSR to solve single-objective problems [156], [157] and multiobjective problems [110], [158], [159], [160]. Instead of starting with uniform distribution, a warm-up mechanism that tunes the initial selection probability distribution was designed for UCB-based OSR in [89], to better select operators in the early stage.

MAB-based methods leverage the developed technologies for solving MAB problems, which demonstrates promising performance under specific assumptions. However, these assumptions might not hold and are expensive to verify in the context of AOS. Modifications and parameter tuning are required to achieve good performance with MAB-based OSR methods for specific problems and meta-heuristics.

5) Subpopulation-Based Selection: In population-based meta-heuristics, the solution population can be divided into several subpopulations, each associated with a specific operator. The solutions in a subpopulation are generated by the corresponding operator. Subpopulation-based methods conduct operator selection by reassigning computational resources between optimization processes of subpopulations, so that the subpopulation with the best operator obtains the most resources to generate new solutions.

Different methods have been proposed for adjusting the number of new solutions generated in each subpopulation, with different tradeoffs between exploration and exploitation [51], [71], [161]. Another approach is to reallocate solutions between subpopulations. Different reassignment methods have been proposed [50], [52], [162], varying in the number of reassigned solutions, reassignment criteria (e.g., the fitness of solutions and subpopulation size) and the selection of destination subpopulations. Similar to the lower bound of selection probability ($P_{\min}$), predefined minimal subpopulation size is commonly used to encourage exploration and avoid the exclusion of an operator.

Subpopulation-based OSR methods are mainly designed specifically for meta-heuristics with population decomposition, in which diverse search directions are maintained with different subpopulations. Although search diversity is improved, these methods do not leverage potential collaboration

between operators effectively, as most solutions may only be manipulated by the single operators corresponding to their respective subpopulations for the entire optimization process.

C. Summary

In stateless AOS, CA evaluates an operator's performance on improving solution quality, and OSR makes a selection decision based on the evaluation. Various CA methods prioritize different factors in finding good solutions, while different OSR strategies balance exploration and exploitation differently considering the changes in operators' performances. Raw-evaluation-based CA methods directly take the fitness of generated solutions as the reward values, but reward an operator even if no improvement in solution quality is achieved.

Improvement-based CA methods, on the other hand, calculate rewards based on the improvement in solution quality. These methods are intuitive and straightforward to implement across most meta-heuristic frameworks and problems, and are the most widely used in the literature. However, directly taking the improvement value as the reward suffers from: 1) diminished improvement; and 2) oscillating reward, as discussed in Section IV-A2. The two issues introduce noise and subsequent extra difficulty for OSR to find the best operator and detect the change of the operators' performance. Reward normalization and considering multi-iteration performance with a sliding window are common strategies to handle the issues, but require more effort in mechanism design and parameter tuning.

Success methods consider only the success of improvement and reward operators with predefined (often binary) values, disregarding the degree of improvement. While this approach addresses the two issues, it disregards the improvement magnitude difference between operators, and underestimates operators capable of achieving large improvements.

Rank-based CA methods effectively address these issues by ranking methods in which new solutions are used as references for each other. However, the dependency of one operator's reward on the performance of others complicates the detection of changes in operator performance. Additionally, the ranking also introduces additional computational resource costs. The above three CA categories are shortsighted as they evaluate operators purely based on immediate performance. An operator with low immediate performance may own the ability to enhance the future search potential, e.g., crossover operators in GAs and perturbation operators in LNSs enhance the global search ability. Potential-based CA methods evaluate the long-term performance of operators, by indicators designed to reflect the long-term potential of the optimization process. As those indicators commonly require high-complexity calculations and a population of solutions, the computation resource cost increases by using potential-based CA.

Different CA methods are suitable for different optimization problems and meta-heuristics. For example, for solving constrained optimization problems, survival methods with stochastic ranking-based solution selection may be more suitable to handle the constraints than improvement-based methods that only consider the objective values. While for multiobjective meta-heuristics based on dominance relations, the success method based on dominance relations may be more practical.

Algorithm 2: Meta-Heuristic Framework With State-Based AOS.

The unique steps of state-based AOS are highlighted in yellow, and the common the step of stateless and state-based AOS are highlighted in gray.

```
1: π ← randomly initialize model
2: if offline train then
3:     π ← offline train model
4: end if
5: x ← initialize solution
6: y ← f(x)                                   ▷ Evaluate solution
7: for t ← 1 to T do
8:     s ← extract state feature
9:     ope* ← π(s)                            ▷ Select operator
10:    x' ← ope*(x)                           ▷ Apply selected operator
11:    y' ← f(x')
12:    if y' < y then
13:        x ← x', y ← y'
14:    end if
15:    if online train then
16:        r ← CA(x, x', y, y')               ▷ Calculate reward by credit assignment
17:        π ← update(π, r, ope*, s)          ▷ Update model
18:    end if
19: end for
20: return x
```

After obtaining the reward values from CA, OSR conducts the operator selection. OSR methods are commonly highly parameterized. The parameters greatly impact the performance of an OSR method. Different OSR methods and the same OSR method with different parameter settings majorly differ in exploration–exploitation balance. When the operators' performance is stable, exploitation is desired. Otherwise, an OSR method needs to allocate sufficient resources to explore for monitoring changes in operators' performance. There are several studies for referring that experimentally compare different CA or OSR methods for some specific meta-heuristics or optimization problems [39], [40], [41], [42], [163], [164], while the general analysis, especially theoretical, is limited.

Stateless AOS methods can be constructed by combining a pair of applicable CA and OSR reviewed above. Most stateless AOS methods are computationally efficient, and all information required for decision-making is collected during the optimization, with the option to include additional domain knowledge of (either or both) the meta-heuristic and the optimization problem if needed.

V. State-Based Adaptive Operator Selection

Different from stateless AOS that selects operators based solely on the historical performance of operators, state-based AOS takes the current optimization state into consideration. State-based AOS captures the mapping between the optimization state and operator selection, commonly with machine learning models. For each operator selection, the selection probability distribution is generated or updated based on the current optimization state, as demonstrated in Algorithm 2, where the steps related to state-based AOS are highlighted.

Following the common paradigms in machine learning, most state-based AOS methods formulate the operator selection

TABLE II
SUMMARY OF PROPOSED STATE FEATURES

Value Type	**Solution Feature** (Section V-A1)	**Problem Feature** (Section V-A2)	**Optimization Process Feature** (Section V-A3)
Discrete	Solution variable values [53], [167], [170], [175], [177]; Solution diversity level [46]; Solution fitness level [46], [193]; Fitness statistics level [179], [180], [181], [194]		Stagnation [195], [196], [197]; Restart [172]; Computational resources consumed/left [182], [194]; Previously used operator [80], [166], [174], [184], [185], [189], [196], [197], [198], [199], [200]; Previous operator performance [174], [186], [187], [191]
Continuous	Solution variable values [176], [190], [201]; Solution diversity [166], [172], [173], [190], [195], [202]; FLA metrics and fitness statistics [113], [114], [165], [168], [169], [172], [173], [182], [183], [188], [196], [197], [203]	Constraints and objective related information [174], [182], [203], [204]	Computational resources consumed/left [80], [179], [180], [181], [195], [196]; Previous operator performance [80], [190]

Note: FLA: fitness landscape analysis.

as: Classification/regression, that take the current state features as input and output the prediction of the currently best operator (i.e., classification) or current performance of each operator (i.e., regression), conducting operator selections as independent prediction processes [113], [165], [166], [167], [168], [169], [170]; or *Markov Decision Process* (MDP) [171], which conducts consecutive selections as a sequence of interdependent decision-makings [46], [80], [172], [173], [174], [175], [176], [177], [178], [179], [180], [181], [182], [183], [184], [185], [186], [187], [188], [189], [190], [191], [192].

Following [46], AOS can be formulated as a MDP, defined as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P} \rangle$. $\mathcal{S}$ represents the set of states. A state $s \in \mathcal{S}$ is the state of the current optimization process, often composed of features that provide sufficient information related to operators' performance, e.g., the locations of incumbent solutions in the solution space. $\mathcal{A}$ represents the set of actions, where each action is an operator. $\mathcal{R} : (\mathcal{S}, \mathcal{A}) \mapsto \mathbb{R}$ is the reward function that evaluates the contribution or utility of applying an operator $a \in \mathcal{A}$ to the search process $s \in \mathcal{S}$. The transition function $P : (\mathcal{S}, \mathcal{A}) \mapsto \mathcal{S}$ defines how the search progresses after applying an operator.

For MDP formulation, reward calculation follows the same conception as CA methods introduced in Section IV-A. For classification or regression formulation, the reward values obtained from CA are used to define the true labels (e.g., the best operator) or the target values, respectively, for training the prediction models. In addition to CA, in both formulations, two key modules exist in state-based AOS: *state feature extraction*, i.e., how to obtain the numerical features to represent the optimization states, and *selection strategy*, i.e., how to construct and utilize the mapping, including a learning model, a model training method and a decision strategy for selecting an operator based on the model. The following sections review the design of state feature extraction and selection strategy.

A. State Feature Extraction

State-based AOS makes decisions based on input state features. To select the best operator accurately, state features must be informative and comprehensive. State features should also be as compact (excluding irrelevant information) as possible so that the learning model can efficiently focus on useful information. Operators are applied directly to solutions. Therefore, state features for operator selection are mostly extracted from features of solutions, in either the decision space or the objective space, namely *solution features* (cf. Section V-A1). Characteristics of the problem to be solved, namely *problem features* (cf. Section V-A2), are also important in operators' performance. Besides, the statistics of the optimization process, namely *optimization process features* (cf. Section V-A3), could provide instruction in operator selection.

Note that the features from different categories can be combined together, forming a hybrid higher-dimensional feature vector, to provide more comprehensive information. Table II summarizes the proposed state features. In the following, the proposed features of each category are reviewed.

1) Solution Features:

a) Decision space solution features: Decision space state features are extracted from the incumbent solutions in decision space, i.e., values of the decision variables. Since operators manipulate the decision variable values, decision space solution features are the most directly related to the operators' performance. The decision variable vectors of the manipulated solutions, typically represented as vectors of continuous values in continuous optimization problems [176], [190], [201] or as binary vectors in assignment problems [53], [167], [170], [175], [177], are directly used as state feature. However, study on permutation problems is relatively limited (e.g., [174]), due to the difficulty in designing learning models to process the sequential nature of solutions.

The distance and diversity between incumbent solutions in decision space are also used as state features in solving continuous problems [166], [173], [190], assignment problems (e.g., quadratic assignment problems (QAPs) [172]), and permutation problems (e.g. vehicle routing problems (VRPs) [202]). The discretization of diversity values, for example, segmenting the diversity range into several predefined levels (e.g., high,

TABLE III
SUMMARY OF LEARNING MODEL USED IN THE LITERATURE

Formulation	**Online-Training** (Section V-B1)	**Offline-Training** (Section V-B2)
Classification/regression	DWM [113], [114], [165]; CART [166]; NN [53], [170]	Decision tree [168]; Random forest [169]; SVM [169]; NN [169]
MDP	Q table (Sarsa [196]; Q-learning [46], [175], [177], [178], [180], [181], [184], [185], [186], [187], [189], [191], [201]); NN (DQN [80], [176]; DDQN [188], [190])	Q table (Q-learning [177], [179], [183]; [194], [200], [214]); NN (Deep Q-learning [172], DQN [182], [199]; DDQN [173], REINFORCE [174], PG [192]; PPO [182], [197], [202], [203], [204])
Others	CBR [167], [215], [216]; Modified stateless methods [193], [195]	

Note: SVM: support vector machine, NN: neural network, DWM: dynamic weighted majority, DQN: deep Q network, DDQN: double deep Q network, PPO: proximal policy optimization, PG: policy gradient, CBR: case-based reasoning.

medium, and low) [46], facilitates the use of models accepting only discrete input values.

b) Objective space solution features: The main goal of applying operators is to improve the objective (fitness) values. Objective space solution features encode the current state directly from the objective values. Various statistics of the objective values in the current population and FLA metrics are used as state features (e.g., [168], [169], [183], [203]) to describe the overall objective space characteristics of multiple incumbent solutions. Methods similar to the discretization of diversity can also convert objective values [46], [193] and the statistics [179], [180], [181], [194] into discrete values.

2) Problem Features: The problem to be solved also impacts the operators' performance. Information of the problem is used as state features, for example, the distances between nodes in routing problems [174], [204], statistics of constraint information [182], [203]. Although the problem information is static during optimization, it helps the model to effectively learn to solve different problems.

3) Optimization Process Features: Statistics of the optimization process are also used as state features, for example, the previously applied operator (e.g., [184], [185], [200]) and their performance (e.g., [186], [187], [191]) stagnation (i.e., the number of iterations without improvement) [195], [196], [197] and computational resources consumed/left (e.g., [80], [195]). These features involve more information related to the problem-solving process like computation sources consumption into the model's decision-making process.

B. Selection Strategy

When the state feature is defined, state-based AOS methods train models that map the state features to the best operator (or the performance of each operator), and then select operators during optimization with the trained models. As state-based AOS can be formulated into classification or regression task or MDP, the model, the training method and the decision strategy based on the model outputs follow the same methods in machine learning studies. For the AOS methods based on classification or regression formulation, supervised learning methods are used for model design, training and utilizing (testing). For example, support vector machine (SVM) [205], decision tree [206] and random forest [207] are used as the model and trained with corresponding training methods. With a trained model, the predicted best operator will be selected during optimization. For the AOS methods based on MDP formulation, the models are usually trained with RL algorithms, e.g., Q-learning [208], deep Q-network (DQN) [209], proximal policy optimization (PPO) [210], double deep Q-network (DDQN) [211], policy gradient (PG) [212] and REINFORCE [213], with corresponding models such as Q-table or neural network (NN) and decision strategy like ϵ-greedy.

Nevertheless, in the context of AOS, a specific question arises regarding the model training: when should data be collected and when should the model be (re-)trained? According to the answer to this question, the state-based AOS model training methods can be divided into the following two classes: Online-training (Section V-B1), and Offline-training (Section V-B2). Table III summarizes the used models and training methods. In the following part of this section, we focus on the question and discuss the two classes respectively.

1) Online-Training Approaches: Similar to stateless AOS, online-training state-based AOS records the information of historical operator applications to instruct the following operator selections during a single optimization process. The AOS model is trained while solving one single problem (cf. Algorithm 2 with only online training).

At each iteration, current state features are extracted, and the operator selection probability distribution is provided by the model with the state features as input. After applying the selected operator, the state features and the rewards obtained are recorded in the memory. Then, the AOS model is trained with the memory. Existing online-training AOS methods differ in the state features and the mapping approach from state features to operator selection probability distributions.

RL methods that map states into acting policy are suitable for operator selection by taking each operator as an action. With discrete state features, Q-learning and its variants are used for online training, where the Q-table is used as the model [184], [214]. The estimated performance of each operator in each possible state is maintained in the Q-table. For each selection, the operator with the highest estimated performance is selected. ϵ-greedy method is commonly embedded to cooperate with

Q-table. A small probability (ϵ) is assigned to uniformly random operator selection to encourage the exploration of the operator's ability. For continuous state features, neural networks work as the model and are trained with the DQN and its variants in [80], [176], [188], [199].

Supervised learning methods are also investigated. To predict the reward of each operator in the current state, the classification and regression trees (CART) [217] and dynamic weighted majority (DWM) [218] were used in [166] and [113], [114], [165], respectively. The operator with the largest predicted reward will be selected.

The work of [167] proposed to select operators with case-based reasoning (CBR) methods. A case memory records all encountered states, the operator applied to each state, and the obtained rewards. For a new state, the operators' performance is estimated based on similar states recorded in the memory, and the operator with the best rewards is selected. For solving scheduling problems, CBR methods are studied to select repair operators for repairing infeasible schedule plans [215], [216].

Some online-training state-based methods are designed by adding state features into stateless AOS methods. The work of [195] assumed operators' performance is linearly related to the state features and incorporated linear transformed state features into the UCB function in a MAB-based stateless AOS method. The study of [193] assumed that the ideal operator selection probabilities differ for solutions with different objective values. Therefore, the study divided the solutions' objective values into multiple levels and ran a stateless AOS method independently at each level. In this method, the stateless AOS method corresponding to the incumbent solution's objective value level selects the operator to apply on it.

In online-training state-based AOS, it is crucial to balance the trade-off of computational resources used between model training/updating and optimization. The existing approaches commonly conduct the tradeoff with predefined fixed parameters, such as a fixed number of training steps taken after a fixed number of iterations. There is a growing need to explore more adaptive control methods, including adaptive parameter settings, as a promising avenue for future research.

2) Offline-Training Approaches: The steps of offline-training methods are similar to online-training methods, with the primary difference being that the model is trained before being applied in the optimization process to solve the target problem that needs to be addressed (cf. Algorithm 2 with offline training). Typically, the model is trained on other training problems before being applied to solve the target problem. In other words, the expensive data collection and model training process is separated from the optimization process of the target problem, increasing the optimization efficiency.

The model training and utilization methods, based on supervised learning or RL methods, in offline-training AOS are similar to the ones in online-training introduced in Section V-B1. Therefore, we primarily focus on the collection of training data and training problems, which are components that need to be designed specifically in offline-training methods.

In offline-training AOS, training data is commonly collected by directly running the meta-heuristic to solve the training problems. How to select operators during solving the training problems is important. Uniformly random selection (e.g., [169]), selection with the partially trained model (e.g., [173], [174], [182], [203]) and repeatedly solving a problem while each time using a single operator (e.g., [168]) are three investigated ways. Selection with the partially trained model is commonly used in the AOS methods based on RL, where data is collected by the policy model interacting with the environment. The other two methods are more commonly used in AOS methods based on supervised learning.

The selection of training problems is also critical in model training. Training problems should be representative of all the target problems to be solved. For studies using existing benchmark problems, a typical way is to classify all problems based on specific features. Then, some problems are selected from each class to form the training set, so that each class can be represented [168], [173], [182], [203], [204]. The remaining problems are used as the test problems, i.e., target problems. If the target problem instance is generated with a specific rule or probability distribution, training problem instances can be generated with the same rule or probability distribution [174]. However, in real-world scenarios where the problems to be solved are unpredictable at the time of training, selecting appropriate training problems becomes more challenging.

C. Summary

By explicitly extracting the state features during optimization, and selecting operators based on the current state, state-based AOS methods can effectively react when the optimization state and the operators' performance consequently change during optimization. Machine learning methods are widely used in state-based AOS methods to learn the mapping from states to operator selections. Ideally, with instructive state features and a well-trained model, a state-based AOS method can tackle the main issue of stateless AOS, i.e., the slow reaction to the rapid changes in operators' performance.

State feature extraction is crucial. Ideally, the decision variable values together with the solved problems contain sufficient information to determine the performance of operators. However, directly and only taking those two items as state features commonly results in poor selection performance, due to the model's limited ability to handle the complexity of the solution space. Adding more information about the optimization process is helpful. The design of the feature extraction method is highly dependent on the used model, meta-heuristic algorithm and optimization problem.

Compared with online-training AOS, offline-training AOS decouples high-complexity data collection and model training from the optimization process of the target problem, increasing the resources allocated to the optimization. In many real-world scenarios the problems encountered are very similar and the experiences obtained during the offline training on seen problems can be very instructive for solving the unseen ones. With extensive offline training on instructive experiences, offline-training methods can outperform online-training methods. However, there are many other cases where the similarity

between problems cannot be guaranteed, for example, the production and logistics scheduling problems in e-commerce may change dramatically over time due to unforeseen factors such as emerging shopping trends, which can disrupt demand patterns. In such cases, model generalization and collection of instructive training data can become serious issues. Determining if the operators will perform similarly on different problems is difficult, and the experiences obtained from training problems may even be misleading.

Operator evaluation is also important. CA methods introduced in Section IV-A are also used in state-based AOS methods. Raw-evaluation [46], [195], variants of FI [172], [173], [175], [176], [177] and success [167], [174], [178], [200] are commonly used as the reward function in MDP. For state-based AOS methods with classification/regression formulation, the rewards obtained by CA methods are used to define true labels/target values for training. Previously obtained rewards are also used as a feature of the state (cf. Section V-A3), forming the input to the model.

An emerging research direction is to combine offline and online training to maximize the utilization of experiences, e.g., train the model firstly on training problems and then update it while solving the target problems. The works of [177], [183] investigated utilizing the offline-trained model as the initial model for online training. A hybrid AOS framework is proposed in [219], combining stateless and state-based methods to conduct offline-online learning more effectively.

VI. Applications of AOS

In the reviewed literature, an AOS method is typically applied to a specific multioperator meta-heuristic algorithm for solving a specific optimization problem. Characteristics of meta-heuristics and problems greatly affect the performance of an AOS method, and consequently the AOS method design. This section summarizes the problems, datasets and meta-heuristics to which the AOS methods are applied in the reviewed literature and discusses how they affect the design of AOS methods. A detailed summary of the most commonly studied problems and meta-heuristics are provided in Table IV.

A. Problems and Datasets

This article categorizes the optimization problems studied in the AOS literature into assignment, permutation, and continuous optimization problems. Although the first two of the three belong to combinatorial optimization problems, they are presented separately because the permutation characteristic highly affects the AOS design (e.g., in diversity measurement).

Three assignment problems, i.e., OneMax, QAP, and knapsack problem, three permutation problems, i.e., job scheduling, traveling salesman problem (TSP) and VRP, the CEC continuous benchmark [229], [230], [231], [232], [233] and BBOB [234] functions are the most studied single-objective problems. DTLZ [237], WFG [241], ZDT [243] and CEC MO benchmarks [238], [239] are the most studied multiobjective problems. Benchmarks of assignment and permutation problems, such as TSPLIB [244], QAPLIB [245], ORLib [246] and the Solomon instances of vehicle routing problem with time windows [247], are used. Besides benchmarks, real-world industry problems are also studied, such as shuttle system design [223], environmental/economic load dispatch [83], protein structure prediction [109], constellation reconfiguration of artificial satellites [95] and software engineering [49], [136], [248].

The difference between optimization problems significantly affects the definition of solution distance and diversity, commonly used in CA methods (Section IV-A) and state features (Section V-A1). Many distance metrics are available for continuous optimization problems, e.g., Euclidean distance and Manhattan distance. As solutions to assignment problems are usually represented as binary vectors, metrics such as Hamming distance are sometimes used. Problem-specific distance metrics have been designed for permutation problems (e.g., [112], [114]) considering the sequential nature of solutions.

The stability of operator performance, and consequently the change of optimal selection probability distribution, also depends on optimization problems. Due to the noncontinuity of assignment and permutation problems, the performances of operators are more unpredictable and unstable, making balancing the exploration and exploitation harder for OSR [138].

In state-based AOS, state features vary for different optimization problems. Decision (i.e., solution) features, such as decision variables and diversity metrics, of different problems differ due to different solution representations. Problem characteristics like constraints significantly affect on AOS method design. Additional effort is required in designing CA methods due to the complexity of solution evaluation introduced by constraints and multiple contradictory objective functions.

B. Meta-Heuristic Algorithms

Meta-heuristics considered in the AOS literature are categorized as *population-based* and *single-solution-based* ones as the existence of solution population greatly affects the design concept. Most of the reviewed works studied population-based meta-heuristics. Specifically, GA and DE for solving single-objective problems and MOEA/D for solving multiobjective problems are the most studied for both stateless and state-based AOS. The population enables the calculation of solution rank and diversity. At each iteration, multiple operators can be applied simultaneously to different incumbent solutions in the population. Therefore multiple samples of operators' performance can be obtained for better operator evaluation.

One key of designing AOS methods for population-based meta-heuristics is the level of operator selection. It can be made at the population level, i.e., in each iteration an operator is used to generate a population of new solutions, or at the single-solution level, i.e., an operator is selected separately for each incumbent solution. For single-solution level selection, the reward and state involve the incumbent(s) and new solution(s) of only the single operator application. For population-level selection, all incumbent and new solutions in the iteration should be considered. Besides, for single-solution level selection, designers must also determine if updating the selection

TABLE IV
SUMMARY OF MOSTLY STUDIED META-HEURISTICS AND PROBLEMS IN THE AOS LITERATURE

Problem			Stateless AOS		State-Based AOS	
			Population-Based	Single-Solution	Population-Based	Single-Solution
Single-Objective	Assignment	SAT	GA [72], [118], [119], [124], [125]			ILS [196]
		Knapsack	GA [126], [220]	LS [70]; ILS [92]	ABC [169], [175], [177], [183]	
		OneMax	GA [20], [21], [52], [63], [64], [73], [105], [123], [126], [221]		ABC [169], [183]	
		QAP	GA [222]; MA [59], [61]	LS [122], [130]; ILS [102]	MA [172], [46]	
		Others	ABC [68], [71]; GA [20], [41], [64], [65], [73], [103] [145], [194], [223]; MA [53]; Social network search [143]	LS [121], [224]; ILS [62]; SA [115]	GA [193]; MA [53]	LS [47]; ILS [134], [170]; Tabu search [167]
	Permutation	Job Scheduling	GA [48], [74]; DE [117]	LS [81]		Iterated greedy [187]; ILS [196]
		TSP	GA [161], [220], [225]	LS [130]	GA [179], [180] [181], [194], [204]	LS [199]; ILS [196]
		VRP	MA [112], [114], [226]	LS [70], [157]; LNS [154]; SA [156]	GA [182], [203]; MA [113], [114], [165], [185]	LS [174], [199], [214]; ILS [196]; SHH [202]
		Others	GA [162], [227], [228]	ILS [148], [93]		
	Continuous	CEC SO benchmarks [229], [230], [231], [232], [233]	DE [98], [99], [100], [116], [128], [129], [133]		DE [168], [173], [192]; PSO [189]	
		BBOB [234]	DE [67], [106], [111]; GA [67]			
		Others	DE [60], [66], [94]; ES [51]; GA [50], [164], [235]; Whale optimization [104]; SHH [201]; PSO [97]; MA [57]		GA [142]	
Multi-objective	Assignment	mQAP	NSGA-III [158]; MOEA/D [236]	LS [137]		
		Others	MA [75], GA [95], [96]		NSGA-II [120], [186]	
	Permutation	Job Scheduling	MA [101]		NSGA-II [184]	
	Continuous	ZDT [237]	MOEA/D [78], [87], [89]		MOEA/D [176], [190], [191]	
		DTLZ [237]	DE [58]; NSGA-III [108], [109]; MOEA/D [39], [77], [87], [89], [191]		ES [198]; MOEA/D [176], [178], [190]	
		CEC MO Benchmarks [238], [239]	MOEA/D [39], [76], [77], [78], [84], [85], [86], [87], [89], [127], [160], [191], [240]; DE [69]; NSGA-III [110]		MOEA/D [166], [176]	
		WFG [241]	NSGA-III [109]; MOEA/D [39], [77], [87], [89], [160]; SHH [132]		MOEA/D [176] [178], [190], [195]	
		Others	MOEA/D [40], [83], [89], [242]; NSGA-III [158]		MOEA/D [166]; MODE [188]	

Note: SAT: boolean satisfiability problem, QAP: quadratic assignment problem, TSP: traveling salesman problem, BBOB: blackbox optimization benchmark, CEC SO/MO benchmarks: IEEE Congress on Evolutionary Computation single/multiple objective benchmarks, VRP: vehicle routing problem, GA: genetic algorithm, LS: local search, ILS: iterated local search, ABC: artificial bee colony, MA: memetic algorithm, SA: simulated annealing, DE: differential evolution, LNS: large neighborhood search, SHH: selective hyper-heuristic, PSO: particle swarm optimization, ES: evolutionary strategy, NSGA: non-dominated sorting genetic algorithm, MOEA/D: multiobjective evolutionary algorithm based on decomposition, MODE: multi-objective differential evolution, mQAP: multi-objective quadratic assignment problem.

probability distribution after each operator application or after all the applications in the current iteration.

C. Guidance for AOS Method Selection

So far we have introduced a variety of AOS methods. Detail comparisons and discussions between each category are presented in Sections IV-C and V-C. Based on the conceptual and empirical evidence from existing studies, this section further provides guidelines on how to select proper AOS methods under different application scenarios.

First, offline-training state-based methods are recommended when enough resources (i.e., computational resources and representative training instances) are available. FLA metrics and optimization process features (e.g., last used operator, last reward and computational resources consumed or left) are recommended as state features, together with FIR or success as the CA method, generally due to their low dependence on meta-heuristic components and problem characteristics. Empirical evidence demonstrates their remarkable capability in representing search state for operator selection [173], [188], [196]. Several works in AOS [182], [194], [199], [214] report the promising performance of Q-learning and DQN in training models with discrete and continuous state features, respectively.

Second, when computational resources or representative training problems are insufficient for offline training, stateless and online-training state-based methods are more practical. If highly instructive state features can be designed based on domain knowledge, online-training state-based methods are recommended. The recommended training algorithms are also Q-learning and DQN. Otherwise, stateless methods are recommended. The most widely studied OSRs, i.e., AP [54] and DMAB [55] are recommended, because of their illustrated favorable and stable performance over different problems and meta-heuristics in literature [61], [63], [67], [69]. FIR and success are still recommended as CA for both conditions.

Besides, as the above recommendation is relatively general, we provide additional suggestions to meet some specific concerns in problems and meta-heuristics, as follows.

Based on the problem to be solved

1) For solving multiobjective problems, success methods based on Pareto dominance [96] and hypervolume improvement [75] are recommended as CA.
2) For solving constrained problems where infeasible solutions may survive, the constraint violation degree of a solution can be added to CA [133] and state features.
3) When the problem-solving is sensitive to computational cost, the cost of each operator application can be added in both CA [134] and state features.
4) For solving problems requiring high global searchability (e.g., multimodel problems), potential-based CAs are recommended.

Based on the meta-heuristic: 1) If the solution population is divided into subpopulations, subpopulation-based OSR is easy to embed. 2) For highly random operators, statistics within a sliding window can be utilized for stabilizing the reward. 3) If an archive of good solutions is maintained, their objective values can be used for reward normalization in improvement-based CA, and the distance between the incumbent and recorded solutions can serve as state features. Given the overlaps in the situations described, users should balance the tradeoffs based on their needs.

VII. Research Challenges

This section discusses the challenges in the AOS study. Subsequent subsections delve into the challenges associated with the selection between stateless and state-based AOS, the study of each type, the combination of both types, the comprehensive performance analysis of AOS methods, and the improvement of scalability, respectively.

A. Selecting Between Stateless and State-Based Methods

The initial challenge lies in determining whether a stateless or state-based AOS method is more suitable. While Section VI-C offers some guidance, further complexities arise when considering the specific meta-heuristic and problem domain. Accurate effectiveness evaluation of the state features design is the major challenge. As the utilization of state features is the crucial difference between stateless and state-based AOS, the evaluation of state features will be a significant reference for selecting between the two categories.

State-based methods are preferred if the state feature is effective in representing the state information about operators' performance. Otherwise, stateless methods are expected to be preferable. With enough computational resources, one can test the state features by running a state-based method to solve the problem. However, for many scenarios when computational resources for sufficient tests are unavailable, more efficient evaluation (or estimation) methods are required.

B. Challenges in Stateless AOS

Stateless AOS methods make decisions based on operators' historical performance recorded during optimization. Three main challenges arise due to the framework of stateless AOS. First, it is hard for CA to accurately evaluate the performance, especially the long-term performance, before the optimization ends. For optimization problems, only the quality of the final solution, obtained after all the computational resource budget is consumed, is concerned. The final solution, and consequently the contribution of each operator on it, cannot be determined before optimization ends.

Second, it is hard to accurately estimate the operators' performance without considering the incumbent solutions. The performance of an operator depends on its operation and the solution it manipulates, while in the stateless AOS, an operator is evaluated alone regardless of the manipulated solutions.

Due to the above challenges, the rewards assigned to operators are unstable and the range of those values varies, as described in Sections IV-A and IV-C. This leads to the third challenge: it is hard for OSR to accurately detect changes in operators' performance. With highly unstable rewards, commonly used stateless AOS methods may fail to detect changes, severely affecting the meta-heuristics' performance [138].

C. Challenges in State-Based AOS

The main challenges of state-based AOS methods are designing informative state features and efficiently training models to map from state to operator selection probability distribution within a limited computational budget. Especially in online-training state-based AOS, a dilemma of computational resource allocation between training and optimization exists.

Offline-training methods face a specific challenge: in many cases, the training and target problems may not be sufficiently similar, resulting in poor performance on the target problems for models that overfit to the training problems. Therefore, the generalization ability of offline-training state-based AOS model is expected. There is also a specific challenge that exists in online-training methods, which is the dilemma between model training and performing meta-heuristic steps, such as generating and evaluating new solutions. Current online-training state-based AOS methods alternate between training and optimization at fixed intervals. Adaptive resource allocation between optimization and training has the potential to improve the efficiency of AOS.

These challenges are also prevalent in the broader field of *learn to optimize* (L2O). A general discussion from the perspective of L2O can be found in the recent review [12].

D. Combining Stateless and State-Based AOS Methods

Stateless AOS methods can make relatively good decisions with a small computational cost. A well-trained state-based AOS method may be more effective by referencing the current state, but the training is expensive. Combining stateless and state-based methods is intuitive, while designing effective approaches that leverage their respective strengths while avoiding their weaknesses is challenging. To achieve good outcomes, the combination needs to prioritize either stateless or state-based AOS when it owns superior performance at any given time. Designing accurate methods for evaluating an AOS method's performance and detecting performance changes has the potential to better the combination. Although a recent work [219] explored this combination and proposed a general framework, more in-depth investigation is needed.

E. Conducting Comprehensive Performance Analysis

Conducting comprehensive experimental performance analysis of AOS methods is both challenging and critical. It is essential for understanding the characteristics of each method, selecting the suitable method for a given scenario, and improving existing methods. However, the diversity of problems and meta-heuristics adds complexity, as the performance of an AOS method can vary significantly across different scenarios. Many AOS methods are parameter sensitive, requiring extensive testing across multiple parameter settings to obtain reliable results. Additionally, the large number of existing AOS methods, coupled with the ability to create more by combining different strategies for each component based on the proposed taxonomy, further complicates the analysis process.

AOS is highly modular where strategies of components can be easily combined. Building a comprehensive library would facilitate experimental analysis. Moreover, it would promote the application of AOS to real-world optimization scenarios.

Theoretical analysis is also important. Existing studies with performance analysis of AOS methods majorly focus on experimental analysis, while theoretical analyses are seldom involved. Theoretical analysis helps to understand the fundamental principles and limitations of an AOS method, offering insights into its characteristics across different scenarios.

F. Improving Scalability

Improving the scalability when evaluating operators is a general challenge in both stateless and state-based AOS methods. With a larger problem size (e.g., number of decision variables), given an operator, commonly more new solutions with different qualities can be generated, making the accurate evaluation of the operator more expensive. CA methods with good scalability can significantly improve problem-solving performance, especially for large-scale problems, by efficiently evaluating operators without excessive computational costs.

VIII. Conclusion

In this article, we provide a formal definition of the adaptive operator selection (AOS) task and a comprehensive survey of AOS studies for meta-heuristics for the first time. From the survey, we find that whether to use the current optimization state as an input to select operators is the main distinguishing feature and, based on that, existing AOS methods are categorized into two classes, stateless and state-based methods. The survey also reveals a quick emergence of state-based methods, especially those based on deep reinforcement learning technologies. We conclude that the traditional component division for AOS [13], namely credit assignment (CA) and operator selection rule (OSR), does not effectively capture the key elements of state-based approaches, especially with respect to state definition and selector model training. Therefore, we introduce a comprehensive taxonomy dividing stateless methods into CA and OSR, and state-based methods into CA, state features, and selection strategy.

This survey also shows that various implementation methods have been proposed for each component, with different approaches focusing on distinct aspects of the optimization process. We organize and categorize these methods, offering an in-depth analysis of their specialties and limitations.

Furthermore, this survey reveals that the design of AOS methods is significantly shaped by the meta-heuristics and optimization problems they are applied to. Consequently, we summarize the meta-heuristics and problems studied in the literature of AOS and provide guidance on selecting suitable AOS methods for specific scenarios. Finally, we identify key challenges that could guide future advancements in AOS.

Acknowledgment

The authors would like to thank all anonymous reviewers for their careful review and insightful comments.

REFERENCES

[1] K. Hussain, M. N. M. Salleh, S. Cheng, and Y. Shi, "Metaheuristic research: A comprehensive survey," *Artif. Intell. Rev.*, vol. 52, no. 4, pp. 2191–2233, 2018.

[2] J. G. C. Costa, Y. Mei, and M. Zhang, "Learning to select initialisation heuristic for vehicle routing problems," in *Genetic Evol. Comput. Conf.* New York, NY: ACM, 2023, pp. 266–274.

[3] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[4] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Int. Conf. Neural Networks*, vol. 4, 1995, pp. 1942–1948.

[5] F. Glover and M. Laguna, *Tabu Search*. New York: Springer, 1998, pp. 2093–2229.

[6] N. Mladenović and P. Hansen, "Variable neighborhood search," *Comput. Oper. Res.*, vol. 24, no. 11, pp. 1097–1100, 1997.

[7] H.-G. Beyer and H.-P. Schwefel, "Evolution strategies—A comprehensive introduction," *Nat. Comput.*, vol. 1, no. 1, pp. 3–52, 2002.

[8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[9] C. Reeves, "Modern heuristic techniques for combinatorial problems," in *Advanced Topics in Computer Science Series*. New York, NY, USA: McGraw-Hill, 1995.

[10] X. Yao, Y. Liu, and G. Lin, "Evolutionary programming made faster," *IEEE Trans. Evol. Computation*, vol. 3, no. 2, pp. 82–102, Feb. 1999.

[11] A. Eiben and J. Smith, *Introduction to Evolutionary Computing*. New York: Springer, 2015.

[12] K. Tang and X. Yao, "Learn to optimize—A brief overview," *Nat. Sci. Rev.*, p. nwae132, 2024.

[13] Á. Fialho, "Adaptive Operator Selection for Optimization," Ph.D. Thesis, Université Paris Sud - Paris XI, 2010.

[14] W. Lan, Z. Ye, P. Ruan, J. Liu, P. Yang, and X. Yao, "Region-focused memetic algorithms with smart initialization for real-world large-scale waste collection problems," *IEEE Trans. Evol. Comput.*, vol. 26, no. 4, pp. 704–718, Apr. 2022.

[15] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *Eur. J. Oper. Res.*, vol. 126, no. 1, pp. 106–130, 2000.

[16] L. Davis, "Adapting operator probabilities in genetic algorithms," in *Proc. Int. Conf. Genetic Algorithms*, 1989, pp. 61–69.

[17] J. J. Grefenstette, "Optimization of control parameters for genetic algorithms," *IEEE Trans. Syst., Man, Cybern.*, vol. 16, no. 1, pp. 122–128, Jan. 1986.

[18] A. L. Tuson and P. Ross, "Adapting operator probabilities in genetic algorithms," Master thesis, Univ. Edinburgh, 1995.

[19] B. A. Julstrom, "What have you done for me lately? Adapting operator probabilities in a steady-state genetic algorithm," in *Proc. Int. Conf. Genetic Algorithms*. San Mateo, CA, USA: Morgan Kaufmann Publishers Inc., 1995, pp. 81–87.

[20] A. Tuson and P. Ross, "Adapting operator settings in genetic algorithms," *Evol. Comput.*, vol. 6, no. 2, pp. 161–184, 1998.

[21] F. Lobo and D. Goldberg, "Decision making in a hybrid genetic algorithm," in *IEEE Int. Conf. Evol. Computation*, 1997, pp. 121–125.

[22] C. Huang, Y. Li, and X. Yao, "A survey of automatic parameter tuning methods for metaheuristics," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 201–216, Feb. 2020.

[23] J. H. Drake, A. Kheiri, E. Özcan, and E. K. Burke, "Recent advances in selection hyper-heuristics," *Eur. J. Oper. Res.*, vol. 285, no. 2, pp. 405–428, Feb. 2020.

[24] X. Yao, "Simulated annealing with extended neighbourhood," *Int. J. Comput. Math.*, vol. 40, nos. 3–4, pp. 169–189, 1991.

[25] X. Yao, "Dynamic neighbourhood size in simulated annealing," in *Int. Joint Conf. Neural Networks*, vol. 1, 1992, pp. 411–416.

[26] K. Tang, Y. Mei, and X. Yao, "Memetic algorithm with extended neighborhood search for capacitated arc routing problems," *IEEE Trans. Evol. Comput.*, vol. 13, no. 5, pp. 1151–1166, 2009.

[27] S. T. W. Mara, R. Norcahyo, P. Jodiawan, L. Lusiantoro, and A. P. Rifai, "A survey of adaptive large neighborhood search algorithms and applications," *Comput. Oper. Res.*, vol. 146, 2022, Art. no. 105903.

[28] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evol. Comput.*, vol. 27, no. 1, pp. 3–45, Jan. 2019.

[29] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "SATzilla: Portfolio-based algorithm selection for SAT," *J. Artif. Intell. Res.*, vol. 32, pp. 565–606, 2008.

[30] J. Liu, "Portfolio methods in uncertain contexts," Ph.D. dissertation, Université Paris-Saclay, 2015.

[31] K. Tang, F. Peng, G. Chen, and X. Yao, "Population-based algorithm portfolios with automated constituent algorithms selection," *Inf. Sci.*, vol. 279, pp. 94–104, 2014.

[32] K. Tang, S. Liu, P. Yang, and X. Yao, "Few-shots parallel algorithm portfolio construction via co-evolution," *IEEE Trans. Evol. Comput.*, vol. 25, no. 3, pp. 595–607, 2021.

[33] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.

[34] S. Liu, Y. Zhang, K. Tang, and X. Yao, "How good is neural combinatorial optimization? A systematic evaluation on the traveling salesman problem," *IEEE Comput. Intell. Mag.*, vol. 18, no. 3, pp. 14–28, Mar. 2023.

[35] M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, and E.-G. Talbi, "Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art," *Eur. J. Oper. Res.*, vol. 296, no. 2, pp. 393–422, 2022.

[36] P. R. d. O. da Costa, J. Rhuggenaath, Y. Zhang, and A. Akcay, "Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning," in *Proc. Asian Conf. Mach. Learn.*, vol. 129. PMLR, 2020, pp. 465–480.

[37] P. Da Costa, Y. Zhang, A. Akcay, and U. Kaymak, "Learning 2-opt local search from heuristics as expert demonstrations," in *Proc. 2021 Int. Joint Conf. Neural Networks (IJCNN)*, 2021, pp. 1–8.

[38] Q. Wang, Y. Hao, and J. Zhang, "Generative inverse reinforcement learning for learning 2-opt heuristics without extrinsic rewards in routing problems," *J. King Saud Univ. - Comput. Inf. Sciences*, vol. 35, no. 9, 2023, Art. no. 101787.

[39] N. Hitomi and D. Selva, "A classification and comparison of credit assignment strategies in multiobjective adaptive operator selection," *IEEE Trans. Evol. Comput.*, vol. 21, no. 2, pp. 294–314, Feb. 2017.

[40] J. A. Brambila-Hernández, M. Á. García-Morales, H. J. Fraire-Huacuja, A. B. del Angel, E. Villegas-Huerta, and R. Carbajal-López, *Experimental Evaluation of Adaptive Operators Selection Methods for the Dynamic Multiobjective Evolutionary Algorithm Based on Decomposition (DMOEA/D)*. New York: Springer, 2023, pp. 307–330.

[41] Á. Fialho, M. Schoenauer, and M. Sebag, "Analysis of adaptive operator selection techniques on the royal road and long K-path problems," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2009, pp. 779–786.

[42] M. Sharma, M. López-Ibáñez, and D. Kazakov, "Unified framework for the adaptive operator selection of discrete parameters," 2020, *arXiv:2005.05613*.

[43] M.-H. Tayarani-N., X. Yao, and H. Xu, "Meta-heuristic algorithms in car engine design: A literature survey," *IEEE Trans. Evol. Comput.*, vol. 19, no. 5, pp. 609–629, May 2015.

[44] Z.-H. Zhan, Z.-J. Wang, H. Jin, and J. Zhang, "Adaptive distributed differential evolution," *IEEE Trans. Cybern.*, vol. 50, no. 11, pp. 4633–4647, Nov. 2020.

[45] S.-H. Wu, Z.-H. Zhan, and J. Zhang, "Safe: Scale-adaptive fitness evaluation method for expensive optimization problems," *IEEE Trans. Evol. Comput.*, vol. 25, no. 3, pp. 478–491, Mar. 2021.

[46] S. D. Handoko, D. T. Nguyen, Z. Yuan, and H. C. Lau, "Reinforcement learning for adaptive operator selection in memetic search applied to quadratic assignment problem," in *Proc. Compan. Publ.ication Conf. Genetic Evol. Comput*. New York: ACM, 2014, pp. 193–194.

[47] J. Santos, A. Ferreira, and G. Flintsch, "An adaptive hybrid genetic algorithm for pavement management," *Int. J. Pavement Eng.*, vol. 20, no. 3, pp. 266–286, 2019.

[48] J. Stanczak, J. Mulawka, and B. Verma, "Genetic algorithms with adaptive probabilities of operator selection," in *Proc. Int. Conf. Comput. Intell. Multimedia Appl.*, 1999, pp. 464–468.

[49] C. Hanna, A. Blot, and J. Petke, "Reinforcement learning for mutation operator selection in automated program repair," 2023, *arXiv:2306.05792*.

[50] R. Thomsen and T. Krink, "Self-adaptive operator scheduling using the religion-based EA," in *Parallel Problem Solving From Nature*. New York: Springer, 2002, pp. 214–223.

[51] Q. T. Pham, "Competitive evolution: A natural approach to operator selection," in *Workshop Evol. Comput.* New York: Springer, 1995, pp. 49–60.

[52] C. Candan, A. Goeffon, F. Lardeux, and F. Saubion, "A dynamic island model for adaptive operator selection," in *Genetic Evol. Comput. Conf.* New York: ACM, 2012, pp. 1253–1260.

[53] C. Grelier, O. Goudet, and J.-K. Hao, "A memetic algorithm with adaptive operator selection for graph coloring," in *Evolutionary Computation in Combinatorial Optimization*. New York: Springer, 2024, pp. 65–80.

[54] D. Thierens, "An adaptive pursuit strategy for allocating operator probabilities," in *Genetic and Evolutionary Computation Conference*. New York: ACM, 2005, pp. 1539–1546.

[55] L. DaCosta, Á. Fialho, M. Schoenauer, and M. Sebag, "Adaptive operator selection with dynamic multi-armed bandits," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2008, pp. 913–920.

[56] M. G. Epitropakis, F. Caraffini, F. Neri, and E. K. Burke, "A separability prototype for automatic memes with adaptive operator selection," in *IEEE Symp. Found. Comput. Intell.*, 2014, pp. 70–77.

[57] Y. S. Ong and A. Keane, "Meta-lamarckian learning in memetic algorithms," *IEEE Trans. Evol. Comput.*, vol. 8, no. 2, pp. 99–110, Feb. 2004.

[58] K. Li, Á. Fialho, and S. Kwong, "Multi-objective differential evolution with adaptive control of parameters and operators," in *Learn. Intell. Optim.* New York: Springer, 2011, pp. 473–487.

[59] G. Francesca, P. Pellegrini, T. Stützle, and M. Birattari, "Off-line and on-line tuning: A study on operator selection for a memetic algorithm applied to the QAP," in *Evolutionary Computation in Combinatorial Optimization*. New York: Springer, 2011, pp. 203–214.

[60] K. Li, R. Wang, S. Kwong, and J. Cao, "Evolving extreme learning machine paradigm with adaptive operator selection and parameter control," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 21, no. 2, pp. 143–154, 2013.

[61] Z. Yuan, S. D. Handoko, D. T. Nguyen, and H. C. Lau, "An empirical study of off-line configuration and on-line adaptation in operator selection," in *Int. Conf. Learn. Intell. Optim.* New York: Springer, 2014, pp. 62–76.

[62] E. Sonuç and E. Özcan, "An adaptive parallel evolutionary algorithm for solving the uncapacitated facility location problem," *Expert Syst. Appl.*, vol. 224, 2023, Art. no. 119956.

[63] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, "Extreme value based adaptive operator selection," in *Parallel Problem Solving From Nature*. New York: Springer, 2008, pp. 175–184.

[64] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, "Dynamic multi-armed bandits and extreme value-based rewards for adaptive operator selection in evolutionary algorithms," in *Int. Conf. Learn. Intell. Optim.* New York: Springer, 2009, pp. 176–190.

[65] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag,, "Analyzing bandit-based adaptive operator selection mechanisms," *Ann. Math. Artif. Intell.*, vol. 60, nos. 1–2, pp. 25–64, 2010.

[66] W. Gong, Á. Fialho, Z. Cai, and H. Li, "Adaptive strategy selection in differential evolution for numerical optimization: An empirical study," *Inf. Sci.*, vol. 181, no. 24, pp. 5364–5386, 2011.

[67] E. Krempser, Á. Fialho, and H. J. C. Barbosa, "Adaptive operator selection at the hyper-level," in *Parallel Problem Solving From Nature*. New York: Springer, 2012, pp. 378–387.

[68] R. Durgut and M. E. Aydin, "Adaptive binary artificial bee colony algorithm," *Appl. Soft Comput.*, vol. 101, 2021, Art. no. 107054.

[69] S. M. Venske, R. A. Gonçalves, and M. R. Delgado, "ADEMO/D: Multiobjective optimization by an adaptive differential evolution algorithm," *Neurocomputing*, vol. 127, pp. 65–77, 2014.

[70] J. A. Soria-Alcaraz, M. A. Sotelo-Figueroa, and A. Espinal, "Statistical comparative between selection rules for adaptive operator selection in vehicle routing and multi-knapsack problems," in *Fuzzy Logic Augmentation of Neural and Optimization Algorithms: Theoretical Aspects and Real Applications.* New York: Springer, 2018, pp. 389–400.

[71] R. Durgut and M. E. Aydin, "Reinforcement learning-based adaptive operator selection," in *Optimization and Learning*. New York: Springer, 2021, pp. 29–41.

[72] J. Maturana, Á. Fialho, F. Saubion, M. Schoenauer, and M. Sebag, "Extreme compass and dynamic multi-armed bandits for adaptive operator selection," in *Proc. IEEE Congr. Evol. Comput.*, 2009, pp. 365–372.

[73] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, "Extreme: Dynamic multi-armed bandits for adaptive operator selection," in *Conf. Companion Genetic Evol. Comput. Conf.: Late Breaking Papers*. New York: ACM, 2009, pp. 2213–2216.

[74] J. Yan and X. Wu, "A genetic based hyper-heuristic algorithm for the job shop scheduling problem," in *Int. Conf. Intell. Human-Mach. Syst. Cybern.*, vol. 1, 2015, pp. 161–164.

[75] B. Li et al., "A multi-objective memetic algorithm with adaptive local search for airspace complexity mitigation," *Swarm Evol. Comput.*, vol. 83, 2023, Art. no. 101400.

[76] K. Li, Á. Fialho, S. Kwong, and Q. Zhang, "Adaptive operator selection with bandits for a multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 18, no. 1, pp. 114–130, Jan. 2014.

[77] L. Dong, Q. Lin, Y. Zhou, and J. Jiang, "Adaptive operator selection with test-and-apply structure for decomposition-based multi-objective optimization," *Swarm Evol. Comput.*, vol. 68, 2022, Art. no. 101013.

[78] Z. Yan, Y. Tan, W. Zheng, L. Meng, and H. Zhang, "Leader recommend operators selection strategy for a multiobjective evolutionary algorithm based on decomposition," *Inf. Sci.*, vol. 550, pp. 166–188, 2021.

[79] A. S. Ferreira, R. A. Goncalves, and A. T. R. Pozo, "A multi-armed bandit hyper-heuristic," in *Proc. Brazilian Conf. Intell. Syst.*, 2015, pp. 13–18.

[80] A. Dantas and A. Pozo, "The impact of state representation on approximate Q-learning for a selection hyper-heuristic," in *Proc. Brazilian Conf. Intell. Syst.* New York: Springer, 2022, pp. 45–60.

[81] I. O. Yilmazlar and M. E. Kurz, "Adaptive local search algorithm for solving car sequencing problem," *J. Manuf. Syst.*, vol. 68, pp. 635–643, 2023.

[82] H. Chen, Y. Tan, Z. Yan, L. Meng, and W. Wan, "Optimal operator selection based on the hybrid operator selection strategy," *Memetic Comput.*, vol. 14, no. 4, pp. 461–476, Apr. 2022.

[83] R. A. Gonçalves, C. P. Almeida, J. N. Kuk, and A. Pozo, "MOEA/D with adaptive operator selection for the environmental/economic dispatch problem," in *Proc. Latin America Congress on Computational Intelligence*, 2015, pp. 1–6.

[84] R. A. Gonçalves, C. P. Almeida, and A. Pozo, "Upper confidence bound (UCB) algorithms for adaptive operator selection in MOEA/D," in *Int. Conf. Evol. Multi-Criterion Optim.* New York: Springer, 2015, pp. 411–425.

[85] Q. Lin et al., "Adaptive composite operator selection and parameter control for multiobjective evolutionary algorithm," *Inf. Sci.*, vol. 339, pp. 332–352, 2016.

[86] R. A. Gonçalves, C. P. de Almeida, S. M. G. S. Venske, M. R. d. B. d. S. Delgado, and A. T. R. Pozo, "A new hyper-heuristic based on a restless multi-armed bandit for multi-objective optimization," in *Proc. Brazilian Conf. Intell. Syst.*, 2017, pp. 390–395.

[87] L. Prestes, M. R. d. B. d. S. Delgado, R. A. Goncalves, C. P. de Almeida, and A. T. Pozo, "A hyper-heuristic in MOEA/D-DRA using the upper confidence bound technique," in *Proc. Brazilian Conf. Intell. Syst.*, 2017, pp. 396–401.

[88] T. N. Ferreira, J. A. P. Lima, A. Strickler, J. N. Kuk, S. R. Vergilio, and A. Pozo, "Hyper-heuristic based product selection for software product line testing," *IEEE Comput. Intell. Mag.*, vol. 12, no. 2, pp. 34–45, Feb. 2017.

[89] L. Prestes, M. R. Delgado, R. Lüders, R. Gonçalves, and C. P. Almeida, "Boosting the performance of MOEA/D-DRA with a multi-objective hyper-heuristic based on irace and UCB method for heuristic selection," in *Proc. IEEE Congr. Evol. Comput.*, 2018, pp. 1–8.

[90] H. Zhang, Q. Chen, B. Xue, W. Banzhaf, and M. Zhang, "Automatically choosing selection operator based on semantic information in evolutionary feature construction," in *Proc. PRICAI 2023: Trends Artif. Intell.* New York: Springer, 2024, pp. 385–397.

[91] R. Bai, E. K. Burke, G. Kendall, and B. McCollum, "A simulated annealing hyper-heuristic for university course timetabling," in *Int. Conf. Pract. Theory Automated Timetabling*, 2006, pp. 345–350.

[92] D. Thierens, "Adaptive operator selection for iterated local search," in *Engineering Stochastic Local Search Algorithms. Designing, Implementing and Analyzing Effective Heuristics*. New York: Springer, 2009, pp. 140–144.

[93] A. Gunawan, H. C. Lau, and K. Lu, "Enhancing local search with adaptive operator ordering and its application to the time dependent orienteering problem," in *Int. Conf. Pract. Theory Automated Timetabling*, 2016, pp. 181–193.

[94] Z. Fan, Z. Wang, Y. Fang, W. Li, Y. Yuan, and X. Bian, "Adaptive recombination operator selection in push and pull search for solving constrained single-objective optimization problems," in *Proc. Int. Conf. Bio-Inspired Comput.: Theories Appl.* New York: Springer, 2018, pp. 355–367.

[95] J. Hu, L. Yang, H. Huang, and Y. Zhu, "Optimal reconfiguration of constellation using adaptive innovation driven multiobjective evolutionary algorithm," *J. Syst. Eng. Electron.*, vol. 32, no. 6, pp. 1527–1538, 2021.

[96] Y. Xue, H. Zhu, J. Liang, and A. Słowik, "Adaptive crossover operator based multi-objective binary genetic algorithm for feature selection in classification," *Knowl.-Based Syst.*, vol. 227, 2021, Art. no. 107218.
[97] A. Hashemi and M. Meybodi, "A note on the learning automata based algorithms for adaptive parameter selection in PSO," *Appl. Soft Comput.*, vol. 11, no. 1, pp. 689–705, Jan. 2011.
[98] K. M. Sallam, S. M. Elsayed, R. A. Sarker, and D. L. Essam, "Differential evolution with landscape-based operator selection for solving numerical optimization problems," in *Proc. Adaptation, Learn. Optim.* New York: Springer, 2017, pp. 371–387.
[99] Z. Hu, W. Gong, W. Pedrycz, and Y. Li, "Deep reinforcement learning assisted co-evolutionary differential evolution for constrained optimization," *Swarm Evol. Comput.*, vol. 83, 2023, Art. no. 101387.
[100] K. Qiao, J. Liang, K. Yu, M. Yuan, B. Qu, and C. Yue, "Self-adaptive resources allocation-based differential evolution for constrained evolutionary optimization," *Knowl.-Based Syst.*, vol. 235, 2022, Art. no. 107653.
[101] R. Li, W. Gong, L. Wang, C. Lu, and X. Zhuang, "Surprisingly popular-based adaptive memetic algorithm for energy-efficient distributed flexible job shop scheduling," *IEEE Trans. Cybern.*, vol. 53, no. 12, pp. 8013–8023, 2023.
[102] M. M. Drugan and E.-G. Talbi, "Adaptive multi-operator metaheuristics for quadratic assignment problems," in *Proc. EVOLVE—A Bridge Between Probability, Set Oriented Numerics, and Evolutionary Computation V*. New York: Springer, 2014, pp. 149–163.
[103] B. A. Julstrom, "Adaptive operator probabilities in a genetic algorithm that applies three operators," in *Symp. Appl. Comput.* New York: ACM, 1997, pp. 233–238.
[104] Y. Huang, Y. Li, Z. Zhang, and Q. Sun, "A novel path planning approach for AUV based on improved whale optimization algorithm using segment learning and adaptive operator selection," *Ocean Eng.*, vol. 280, 2023, Art. no. 114591.
[105] Á. Fialho, M. Schoenauer, and M. Sebag, "Toward comparison-based adaptive operator selection," in *Genetic Evol. Comput. Conf.* New York: ACM, 2010, pp. 767–774.
[106] Á. Fialho, R. Ros, M. Schoenauer, and M. Sebag, "Comparison-based adaptive strategy selection with bandits in differential evolution," in *Parallel Problem Solving From Nature*. New York: Springer, 2010, pp. 194–203.
[107] M. Pacula, J. Ansel, S. Amarasinghe, and U.-M. O'Reilly, "Hyper-parameter tuning in bandit-based adaptive operator selection," in *Eur. Conf. Appl. Evol. Comput*. New York: Springer, 2012, pp. 73–82.
[108] R. A. Gonçalves, C. P. Almeida, L. M. Pavelski, S. M. Venske, J. N. Kuk, and A. T. Pozo, "Adaptive operator selection in NSGA-III," in *Proc. Brazilian Conf. Intell. Syst.*, 2016, pp. 181–186.
[109] R. A. Gonçalves, L. M. Pavelski, C. P. de Almeida, J. N. Kuk, S. M. Venske, and M. R. Delgado, "Adaptive operator selection for many-objective optimization with NSGA-III," in *Int. Conf. Evol. Multi-Criterion Optim.* New York: Springer, 2017, pp. 267–281.
[110] J. Kuk, R. Goncalves, C. Almeida, S. Venske, and A. Pozo, "A new adaptive operator selection for NSGA-III applied to CEC 2018 many-objective benchmark," in *Proc. Brazilian Conf. Intell. Syst.*, 2018, pp. 7–12.
[111] M. Sharma, M. López-Ibáñez, and D. Kazakov, "Performance assessment of recursive probability matching for adaptive operator selection in differential evolution," in *Parallel Problem Solving from Nature*. New York: Springer, 2018, pp. 321–333.
[112] P. Consoli and X. Yao, "Diversity-driven selection of multiple crossover operators for the capacitated arc routing problem," in *Proc. Evolutionary Computation in Combinatorial Optimisation*. New York: Springer, 2014, pp. 97–108.
[113] P. A. Consoli, Y. Mei, L. L. Minku, and X. Yao, "Dynamic selection of evolutionary operators based on online learning and fitness landscape analysis," *Soft Comput.*, vol. 20, no. 10, pp. 3889–3914, Oct. 2016.
[114] P. A. Consoli, "Adaptive operator search for the capacitated arc routine problem," Ph.D. dissertation, Univ. of Birmingham, 2018.
[115] R. Bai, J. Blazewicz, E. K. Burke, G. Kendall, and B. McCollum, "A simulated annealing hyper-heuristic methodology for flexible decision support," *4OR*, vol. 10, no. 1, pp. 43–66, Jan. 2011.
[116] K. M. Sallam, S. M. Elsayed, R. K. Chakrabortty, and M. J. Ryan, "Improved multi-operator differential evolution algorithm for solving unconstrained problems," in *Proc. IEEE Congr. Evol. Comput.*, 2020, pp. 1–8.
[117] K. M. Sallam, R. K. Chakrabortty, and M. J. Ryan, "A two-stage multi-operator differential evolution algorithm for solving resource constrained project scheduling problems," *Future Gener. Comput. Syst.*, vol. 108, pp. 432–444, 2020.
[118] J. Maturana, F. Lardeux, and F. Saubion, "Autonomous operator management for evolutionary algorithms," *J. Heuristics*, vol. 16, no. 6, pp. 881–909, 2010.
[119] J. Maturana, Á. Fialho, F. Saubion, M. Schoenauer, F. Lardeux, and M. Sebag, "Adaptive operator selection and management in evolutionary algorithms," in *Autonomous Search*. New York: Springer, 2012, pp. 161–189.
[120] H. L. J. Filho, T. N. Ferreira, and S. R. Vergilio, "Incorporating user preferences in a software product line testing hyper-heuristic approach," in *IEEE Congr. Evol. Comput.*, 2018, pp. 1–8.
[121] N. Veerapen, Y. Hamadi, and F. Saubion, "Using local search with adaptive operator selection to solve the progressive party problem," in *IEEE Congr. Evol. Comput.*, 2013, pp. 554–561.
[122] N. Veerapen, J. Maturana, and F. Saubion, "A comparison of operator utility measures for on-line operator selection in local search," in *Int. Conf. Learn. Intell. Optim.* New York: Springer, 2012, pp. 497–502.
[123] J. Belluz, M. Gaudesi, G. Squillero, and A. Tonda, "Operator selection using improved dynamic multi-armed bandit," in *Genetic Evol. Comput. Conf.* New York: ACM, 2015, pp. 1311–1317.
[124] J. Maturana and F. Saubion, "A compass to guide genetic algorithms," in *Parallel Problem Solving from Nature*. Springer, 2008, pp. 256–265.
[125] G. di Tollo, F. Lardeux, J. Maturana, and F. Saubion, "An experimental study of adaptive control for evolutionary algorithms," *Appl. Soft Comput.*, vol. 35, pp. 359–372, 2015.
[126] J. A. Soria Alcaraz, G. Ochoa, M. Carpio, and H. Puga, "Evolvability metrics in adaptive operator selection," in *Genetic Evol. Computation Conf.* ACM, 2014, pp. 1327–1334.
[127] J. Kuk, R. Goncalves, and A. Pozo, "Combining fitness landscape analysis and adaptive operator selection in multi and many-objective optimization," in *Brazilian Conf. Intell. Syst.*, 2019, pp. 503–508.
[128] K. M. Sallam, S. M. Elsayed, R. A. Sarker, and D. L. Essam, "Landscape-assisted multi-operator differential evolution for solving constrained optimization problems," *Expert Syst. Appl.*, vol. 162, 2020, Art. no. 113033.
[129] K. M. Sallam, S. M. Elsayed, R. A. Sarker, and D. L. Essam,, "Landscape-based adaptive operator selection mechanism for differential evolution," *Inf. Sci.*, vols. 418–419, pp. 383–404, 2017.
[130] N. Veerapen, J. Maturana, and F. Saubion, "An exploration-exploitation compromise-based adaptive operator selection for local search," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2012, pp. 1277–1284.
[131] G. Guizzo, G. M. Fritsche, S. R. Vergilio, and A. T. R. Pozo, "A hyper-heuristic for the multi-objective integration and test order problem," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2015, pp. 1343–1350.
[132] S. Zhang, T. Yang, J. Liang, and C. Yue, "A novel adaptive bandit-based selection hyper-heuristic for multiobjective optimization," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 53, no. 12, pp. 7693–7706, Dec. 2023.
[133] S. M. Elsayed, R. A. Sarker, and D. L. Essam, "Multi-operator based evolutionary algorithms for solving constrained optimization problems," *Comput. Oper. Res.*, vol. 38, no. 12, pp. 1877–1896, 2011.
[134] A. Gretsista and E. K. Burke, "An iterated local search framework with adaptive operator selection for nurse rostering," in *Int. Conf. Learn. Intell. Optim.* New York: Springer, 2017, pp. 93–108.
[135] P. Cowling, G. Kendall, and E. Soubeiga, "A hyperheuristic approach to scheduling a sales summit," in *Pract. Theory Automated Timetabling III*. Berlin, Heidelberg: Springer, 2001, pp. 176–190.
[136] K. Z. Zamli, B. Y. Alkazemi, and G. Kendall, "A tabu search hyper-heuristic strategy for t-way test suite generation," *Appl. Soft Comput.*, vol. 44, pp. 57–74, 2016.
[137] M. M. Drugan and D. Thierens, "Generalized adaptive pursuit algorithm for genetic Pareto local search algorithms," in *Genetic Evol. Comput. Conf.* New York: ACM, 2011, pp. 1963–1970.
[138] J. Pei, Y. Mei, J. Liu, and X. Yao, "An investigation of adaptive operator selection in solving complex vehicle routing problem," in *Proc. Pacific Rim Int. Conf. Artif. Intell.* New York: Springer, 2022, pp. 562–573.
[139] T. Runarsson and X. Yao, "Stochastic ranking for constrained evolutionary optimization," *IEEE Trans. Evol. Comput.*, vol. 4, no. 3, pp. 284–294, 2000.
[140] K. M. Malan and A. P. Engelbrecht, "A survey of techniques for characterising fitness landscapes and some possible ways forward," *Inf. Sci.*, vol. 241, pp. 148–163, 2013.
[141] Y. Borenstein and R. Poli, "Information landscapes," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2005, pp. 1515–1522.

[142] C. Xu, Y. Liu, P. Li, and Y. Yang, "Unified multi-objective mapping for network-on-chip using genetic-based hyper-heuristic algorithms," *IET Comput. Digit. Techn.*, vol. 12, no. 4, pp. 158–166, 2018.

[143] K. Z. Zamli, H. S. Alhadawi, and F. Din, "Utilizing the roulette wheel based social network search algorithm for substitution box construction and optimization," *Neural Comput. Appl.*, vol. 35, no. 5, pp. 4051–4071, 2022.

[144] D. E. Goldberg, "Probability matching, the magnitude of reinforcement, and classifier system bidding," *Mach. Learn.*, vol. 5, no. 4, pp. 407–425, 1990.

[145] G. Filograsso and G. di Tollo, "Adaptive evolutionary algorithms for portfolio selection problems," *Comput. Manage. Sci.*, vol. 20, no. 1, 2023.

[146] E. K. Burke, M. Gendreau, G. Ochoa, and J. D. Walker, "Adaptive iterated local search for cross-domain optimisation," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2011, pp. 1987–1994.

[147] S. Zhang, Z. Ren, C. Li, and J. Xuan, "A perturbation adaptive pursuit strategy based hyper-heuristic for multi-objective optimization problems," *Swarm Evol. Comput.*, vol. 54, 2020, Art. no. 100647.

[148] A. Gunawan, H. C. Lau, and K. Lu, "Adopt: Combining parameter tuning and adaptive operator ordering for solving a class of orienteering problems," *Comput. Ind. Eng.*, vol. 121, pp. 82–96, 2018.

[149] T. Lai and H. Robbins, "Asymptotically efficient adaptive allocation rules," *Adv. Appl. Math.*, vol. 6, no. 1, pp. 4–22, 1985.

[150] A. Garivier and E. Moulines, "On upper-confidence bound policies for non-stationary bandit problems," 2008, *arXiv:0805.3415*.

[151] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. Int. Conf. World Wide Web*. New York: ACM, 2010, pp. 661–670.

[152] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, vol. 47, nos. 2/3, pp. 235–256, 2002.

[153] D. V. Hinkley, "Inference about the change-point from cumulative sum tests," *Biometrika*, vol. 58, no. 3, pp. 509–523, 1971.

[154] P. Kalatzantonakis, A. Sifaleras, and N. Samaras, "A reinforcement learning-variable neighborhood search method for the capacitated vehicle routing problem," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 118812.

[155] S. Agrawal and N. Goyal, "Analysis of Thompson sampling for the multi-armed bandit problem," in *Conf. Learn. Theory*, vol. 23. PMLR, 2012, pp. 39.1–39.26.

[156] E. Rodríguez-Esparza, A. D. Masegosa, D. Oliva, and E. Onieva, "A new hyper-heuristic based on adaptive simulated annealing and reinforcement learning for the capacitated electric vehicle routing problem," 2022, *arXiv:2206.03185*.

[157] F. Lagos and J. Pereira, "Multi-armed bandit-based hyper-heuristics for combinatorial optimization problems," *Eur. J. Oper. Res.*, vol. 312, no. 1, pp. 70–91, 2024.

[158] B. N. K. Senzaki, S. M. Venske, and C. P. Almeida, "Hyper-heuristic based NSGA-III for the many-objective quadratic assignment problem," in *Proc. Brazilian Conf. Intell. Syst.* New York: Springer, 2021, pp. 170–185.

[159] N. Gupta, O.-C. Granmo, and A. Agrawala, "Thompson sampling for dynamic multi-armed bandits," in *Int. Conf. Mach. Learn. Appl. Workshops*, vol. 1, 2011, pp. 484–489.

[160] L. Sun and K. Li, "Adaptive operator selection based on dynamic Thompson sampling for MOEA/D," in *Parallel Problem Solving From Nature*. New York: Springer, 2020, pp. 271–284.

[161] W. Boomsma, "A comparison of adaptive operator scheduling methods on the traveling salesman problem," in *Evolutionary Computation in Combinatorial Optimization*. New York: Springer, 2004, pp. 31–40.

[162] J. A. Soria-Alcaraz, G. Ochoa, A. Göeffon, F. Lardeux, and F. Saubion, "Combining mutation and recombination to improve a distributed model of adaptive operator selection," in *Proc. Int. Conf. Artif. Evolution*. New York: Springer, 2016, pp. 97–108.

[163] A. Goëffon, F. Lardeux, and F. Saubion, "Simulating non-stationary operators in search algorithms," *Appl. Soft Comput.*, vol. 38, pp. 257–268, 2016.

[164] J. M. Whitacre, T. Q. Pham, and R. A. Sarker, "Use of statistical outlier detection method in adaptive evolutionary algorithms," in *Proc. Genetic Evol. Comput. Conf.* ACM, 2006, pp. 1345–1352.

[165] P. A. Consoli, L. L. Minku, and X. Yao, "Dynamic selection of evolutionary algorithm operators based on online learning and fitness landscape metrics," in *Simulated Evolution Learn.* New York: Springer, 2014, pp. 359–370.

[166] H. Geng, K. Xu, Y. Zhang, and Z. Zhou, "A classification tree and decomposition based multi-objective evolutionary algorithm with adaptive operator selection," *Complex Intell. Syst.*, vol. 9, no. 1, pp. 579–596, 2022.

[167] S. Grolimund and J.-G. Ganascia, "Driving tabu search with case-based reasoning," *Eur. J. Oper. Res.*, vol. 103, no. 2, pp. 326–338, 1997.

[168] S. Li, W. Li, J. Tang, and F. Wang, "A new evolving operator selector by using fitness landscape in differential evolution algorithm," *Inf. Sci.*, vol. 624, pp. 709–731, 2023.

[169] M. E. Aydin, R. Durgut, A. Rakib, and H. Ihshaish, "Feature-based search space characterisation for data-driven adaptive operator selection," *Evolving Syst.*, vol. 15, no. 1, pp. 99–114, 2023.

[170] C. Grelier, O. Goudet, and J.-K. Hao, "Monte Carlo tree search with adaptive simulation: A case study on weighted vertex coloring," in *Evol. Comput. Combinatorial Optim.* New York: Springer, 2023, pp. 98–113.

[171] M. L. Puterman, "Markov decision processes: Discrete stochastic dynamic programming," Hoboken, NJ, USA: John Wiley, 1994.

[172] T.-H. Teng, S. D. Handoko, and H. C. Lau, "Self-organizing neural network for adaptive operator selection in evolutionary search," in *Int. Conf. Learn. Intell. Optim.* Springer, 2016, pp. 187–202.

[173] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, "Deep reinforcement learning based parameter control in differential evolution," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2019, pp. 709–717.

[174] H. Lu, X. Zhang, and S. Yang, "A learning-based iterative method for solving vehicle routing problems," in *Int. Conf. Learn. Represent.*, 2020.

[175] R. Durgut, M. E. Aydin, and I. Atli, "Adaptive operator selection with reinforcement learning," *Inf. Sci.*, vol. 581, pp. 773–790, 2021.

[176] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 4, pp. 1051–1064, 2023.

[177] R. Durgut, M. E. Aydin, and A. Rakib, "Transfer learning for operator selection: A reinforcement learning approach," *Algorithms*, vol. 15, no. 1, 2022.

[178] K. Jiao, J. Chen, B. Xin, and L. Li, "A reference vector based multiobjective evolutionary algorithm with Q-learning for operator adaptation," *Swarm Evol. Comput.*, vol. 76, 2023, p. 101225.

[179] J. E. Pettinger and R. M. Everson, "Controlling genetic algorithms with reinforcement learning," in *Proc. Genetic Evol. Comput. Conf.* San Mateo, CA, USA: Morgan Kaufmann Publishers Inc., 2002, p. 692.

[180] F. Chen, Y. Gao, Z. qian Chen, and S. fu Chen, "SCGA: Controlling genetic algorithms with Sarsa(0)," in *Proc. Int. Conf. Comput. Intell. Modelling, Control Automat. Int. Conf. Intell. Agents, Web Technologies Internet Commerce*, vol. 1, 2005, pp. 1177–1183.

[181] Y. Sakurai, K. Takada, T. Kawabe, and S. Tsuruta, "A method to control parameters of evolutionary algorithms by using reinforcement learning," in *Int. Conf. Signal-Image Technol. Internet Based Syst.*, 2010, pp. 74–79.

[182] W. Yi, R. Qu, L. Jiao, and B. Niu, "Automated design of metaheuristics using reinforcement learning within a novel general search framework," *IEEE Trans. Evol. Comput.*, vol. 27, no. 4, pp. 1072–1084, 2023.

[183] M. E. Aydin, R. Durgut, and A. Rakib, "Adaptive operator selection utilising generalised experience," 2023, *arXiv: 401.05350*.

[184] P. Li, Q. Xue, Z. Zhang, J. Chen, and D. Zhou, "Multi-objective energy-efficient hybrid flow shop scheduling using Q-learning and GVNS driven NSGA-II," *Comput. Oper. Res.*, vol. 159, 2023, Art. no. 106360.

[185] S. T. Windras Mara, R. Sarker, D. Essam, and S. Elsayed, "Electric vehicle-drone routing problem with optional drone availability," in *Proc. Int. Conf. Intell. Transp. Syst.*, 2023, pp. 827–832.

[186] Y. Ling and L. Pan, "The container stowage planning problem in a barge convoy system via Q-learning-based NSGA-II algorithm," in *Chin. Control Decis. Conf.*, 2023, pp. 4492–4497.

[187] M. Karimi-Mamaghan, M. Mohammadi, B. Pasdeloup, and P. Meyer, "Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem," *Eur. J. Oper. Res.*, vol. 304, no. 3, pp. 1296–1330, 2023.

[188] T. Li, Y. Meng, and L. Tang, "Scheduling of continuous annealing with a multi-objective differential evolution algorithm based on deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1767–1780, Apr. 2024.

[189] W. Li, P. Liang, B. Sun, Y. Sun, and Y. Huang, "Reinforcement learning-based particle swarm optimization with neighborhood differential mutation strategy," *Swarm Evol. Comput.*, vol. 78, 2023, Art. no. 101274.
[190] S. Yin and Z. Xiang, "Adaptive operator selection with dueling deep Q-network for evolutionary multi-objective optimization," *Neurocomputing*, vol. 581, 2024, Art. no. 127491.
[191] J. A. Brambila-Hernández, M. Á. García-Morales, H. J. Fraire-Huacuja, L. Cruz-Reyes, and J. Frausto-Solís, "Novel decomposition-based multi-objective evolutionary algorithm using reinforcement learning adaptive operator selection (MOEA/D-QL)," in *New Horizons for Fuzzy Logic, Neural Networks and Metaheuristics*. New York: Springer, 2024, pp. 149–165.
[192] H. Zhang, J. Sun, T. Bäck, and Z. Xu, "Learning to select the recombination operator for derivative-free optimization," *Sci. China Math.*, vol. 67, no. 6, pp. 1457–1480, 2024.
[193] K. Zhang, T. Weise, and J. Li, "Fitness level based adaptive operator selection for cutting stock problems with contiguity," in *IEEE Congr. Evol. Comput.*, 2014, pp. 2539–2546.
[194] A. Buzdalova, V. Kononov, and M. Buzdalov, "Selecting evolutionary operators using reinforcement learning: Initial explorations," in *Companion Publication of Genetic and Evolutionary Computation Conference*. New York: ACM, 2014, pp. 1033–1036.
[195] R. Gonçalves, C. Almeida, R. Lüders, and M. Delgado, "A new hyper-heuristic based on a contextual multi-armed bandit for many-objective optimization," in *IEEE Congr. Evol. Comput.*, 2018, pp. 1–8.
[196] L. Kletzander and N. Musliu, "Large-state reinforcement learning for hyper-heuristics," *AAAI Conf. Artif. Intell.*, vol. 37, no. 10, pp. 12444–12452, 2023.
[197] J. V. Kallestad, "Developing an intelligent hyperheuristic for combinatorial optimization problems using deep reinforcement learning," Master thesis, Univ. Bergen, 2021.
[198] K. McClymont and E. C. Keedwell, "Markov chain hyper-heuristic (MCHH): An online selective hyper-heuristic for multi-objective continuous problems," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2011, pp. 2003–2010.
[199] A. Dantas, A. F. D. Rego, and A. Pozo, "Using deep Q-network for selection hyper-heuristics," in *Genetic and Evolutionary Computation Conference Companion*. New York: ACM, 2021, pp. 1488–1492.
[200] B. S. Ahmed, E. Enoiu, W. Afzal, and K. Z. Zamli, "An evaluation of Monte Carlo-based hyper-heuristic for interaction testing of industrial embedded software applications," *Soft Comput.*, vol. 24, no. 18, pp. 13929–13954, 2020.
[201] S. M. Bozorgi, S. Yazdani, M. Golsorkhtabaramiri, and S. Adabi, "A hyper-heuristic approach based on adaptive selection operator and behavioral schema for global optimization," *Soft Comput.*, vol. 27, no. 22, pp. 16759–16808, 2023.
[202] W. Qin, Z. Zhuang, Z. Huang, and H. Huang, "A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem," *Comput. Ind. Eng.*, vol. 156, 2021, Art. no. 107252.
[203] W. Yi, R. Qu, and L. Jiao, "Automated algorithm design using proximal policy optimisation with identified features," *Expert Syst. Appl.*, vol. 216, 2023, Art. no. 119461.
[204] J. Schuchardt, V. Golkov, and D. Cremers, "Learning to evolve," 2019, *arXiv:1905.03389*.
[205] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
[206] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.
[207] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
[208] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, nos. 3–4, pp. 279–292, 1992.
[209] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[210] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*.
[211] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016.
[212] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12. Cambridge, MA, USA: MIT Press, 1999.
[213] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, 1992.
[214] Y.-e. Hou, W. Gu, C. Wang, K. Yang, and Y. Wang, "A selection hyper-heuristic based on Q-learning for school bus routing problem," *IAENG Int. J. Appl. Math.*, vol. 52, no. 4, pp. 1–9, 2022.
[215] K. Miyashita and K. Sycara, "CABINS: A framework of knowledge acquisition and iterative revision for schedule improvement and reactive repair," *Artif. Intell.*, vol. 76, nos. 1–2, pp. 377–426, 1995.
[216] S. Petrovic, G. Beddoe, and G. V. Berghe, "Case-based reasoning in employee rostering: Learning repair strategies from domain experts," *Tech. Rep., Automated Scheduling Optimisation and Planning Research Group School of Computer Science and Information Technology*, 2002.
[217] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
[218] J. Kotler and M. Maloof, "Dynamic weighted majority: A new ensemble method for tracking concept drift," in *IEEE Int. Conf. Data Mining*, 2003, pp. 123–130.
[219] J. Pei, J. Liu, and Y. Mei, "Learning from offline and online experiences: A hybrid adaptive operator selection framework," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2024, pp. 1017–1025.
[220] C. Breitschopf, G. Blaschek, and T. Scheidl, "A comparison of operator selection strategies in evolutionary optimization," in *Proc. Int. Conf. Inf. Reuse Integration*, 2006, pp. 136–140.
[221] D. Thierens, "On benchmark properties for adaptive operator selection," in *Conf. Companion Genetic Evol. Comput. Conf.: Late Breaking Papers*. New York: ACM, 2009, pp. 2217–2218.
[222] M. Misir, "Algorithm selection on adaptive operator selection: A case study on genetic algorithms," in *Int. Conf. Learn. Intell. Optim.* New York: Springer, 2021, pp. 237–251.
[223] J. Xiong, B. Chen, Z. He, W. Guan, and Y. Chen, "Optimal design of community shuttles with an adaptive-operator-selection-based genetic algorithm," *Transp. Res. Part C: Emerg. Technol.*, vol. 126, 2021, Art. no. 103109.
[224] S. M. Pour, J. H. Drake, and E. K. Burke, "A choice function hyper-heuristic framework for the allocation of maintenance tasks in danish railways," *Comput. Oper. Res.*, vol. 93, pp. 15–26, 2018.
[225] W. Boomsma, "Using adaptive operator scheduling on problem domains with an operator manifold: Applications to the travelling salesman problem," in *IEEE Congr. Evol. Comput.*, vol. 2, 2003, pp. 1274–1279.
[226] J. Pei, H. Tong, J. Liu, Y. Mei, and X. Yao, "Local optima correlation assisted adaptive operator selection," in *Genetic and Evolutionary Computation Conference*. New York: ACM, 2023, pp. 339–347.
[227] X. Wu, P. Consoli, L. Minku, G. Ochoa, and X. Yao, "An evolutionary hyper-heuristic for the software project scheduling problem," in *Parallel Problem Solving from Nature*. New York: Springer, 2016, pp. 37–47.
[228] S. M. Ahmadi, H. Kebriaei, and H. Moradi, "Adaptive operator selection for path planning in static environments," in *Proc. Intell. Syst. Conf.*, 2015, pp. 285–289.
[229] J. J. Liang, B. Y. Qu, and P. N. Suganthan, "Problem definitions and evaluation criteria for the CEC 2014 special session and competition on single objective real-parameter numerical optimization," Computational Intelligence Laboratory, Zhengzhou University, China and Nanyang Technological University, Singapore, Tech. Rep., 2013.
[230] P. N. Suganthan et al., "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Nanyang Technological University Singapore, Tech. Rep., 2005.
[231] R. Mallipeddi and P. N. Suganthan, "Problem definitions and evaluation criteria for the CEC 2010 competition on constrained real-parameter optimization," Nanyang Technological University, Singapore, Tech. Rep., 2010.
[232] N. Awad, M. Ali, J. Liang, B. Qu, and P. Suganthan, "Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound constrained real-parameter numerical optimization," Nanyang Technological University, Singapore, Tech. Rep., 2016.
[233] C. Yue et al., "Problem definitions and evaluation criteria for the CEC 2020 special session and competition on single objective bound constrained numerical optimization," Computational Intelligence Laboratory, Zhengzhou University, China, Tech. Rep., 2019.
[234] N. Hansen et al., "Comparing continuous optimizers: Numbbo/coco on github," 2019. [Online]. Available: https://doi.org/10.5281/zenodo.2594848

[235] J. M. Whitacre, T. Q. Pham, and R. A. Sarker, "Credit assignment in adaptive evolutionary algorithms," in *Proc. Genetic Evol. Comput. Conf.* New York: ACM, 2006, pp. 1353–1360.

[236] B. N. K. Senzaki, S. M. Venske, and C. P. Almeida, "Multi-objective quadratic assignment problem: An approach using a hyper-heuristic based on the choice function," in *Proc. Brazilian Conf. Intell. Syst.* Springer, 2020, pp. 136–150.

[237] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable test problems for evolutionary multiobjective optimization," in *Evol. Multiobjective Optim.: Theor. Adv. Appl*. New York: Springer, 2005, pp. 105–145.

[238] Q. Zhang, A. Zhou, S. Zhao, P. N. Suganthan, W. Liu, and S. Tiwari, "Multiobjective optimization test instances for the CEC 2009 special session and competition," University of Essex, Tech. Rep., 2008.

[239] R. Cheng et al., "Benchmark functions for the CEC'2018 competition on many-objective optimization," University of Birmingham, Tech. Rep., 2018.

[240] R. A. Gonçalves, J. N. Kuk, C. P. Almeida, and S. M. Venske, "Decomposition based multiobjective hyper heuristic with differential evolution," in *Comput. Collective Intell*. New York: Springer, 2015, pp. 129–138.

[241] S. Huband, P. Hingston, L. Barone, and L. While, "A review of multiobjective test problems and a scalable test problem toolkit," *IEEE Trans. Evol. Comput.*, vol. 10, no. 5, pp. 477–506, 2006.

[242] J. Verheul, "The influence of using adaptive operator selection in a multiobjective evolutionary algorithm based on decomposition," Master thesis, Utrecht University, 2020.

[243] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evol. Comput.*, vol. 8, no. 2, pp. 173–195, 2000.

[244] G. Reinelt, "TSPLIB—A traveling salesman problem library," *ORSA J. Comput.*, vol. 3, no. 4, pp. 376–384, 1991.

[245] R. E. Burkard, S. E. Karisch, and F. Rendl, "QAPLIB—A quadratic assignment problem library," *J. Global Optim.*, vol. 10, no. 4, pp. 391–403, 1997.

[246] P. Chu and J. Beasley, "A genetic algorithm for the multidimensional knapsack problem," *J. Heuristics*, vol. 4, no. 1, pp. 63–86, 1998.

[247] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Oper. Res.*, vol. 35, no. 2, pp. 254–265, 1987.

[248] R. Sulaiman, D. Jawawi, and S. Halim, "Cost-effective test case generation with the hyper-heuristic for software product line testing," *Adv. Eng. Softw.*, vol. 175, 2023, Art. no. 103335.